Wazuh FIM - File Integrity Monitoring
The Advanced Defender's Handbook

Wazuh 4.14.8 (4.14.x series) | syscheck: scheduled, realtime e whodata | Windows e Linux

Autor: Kislley Rodrigues | Wazuh Ambassador Program | Outubro 2026
GitHub: https://github.com/mym0us3r/wazuh-fim-adh

Classificação	**Handbook técnico de Detecção e Resposta com FIM no Wazuh**
Versão de referência	Wazuh Server e agentes 4.14.8 (série 4.14.x)
Plataformas	Windows 11 Pro (LABDESK) | Ubuntu 24.04.5 (labnix) | Kali Linux (pentlab)
Conteúdo	11 regras customizadas | 15 achados técnicos | MITRE ATT&CK | Compliance
Evidências	Capturas reais do Wazuh Discover e do painel FIM Alert Investigation (IA Assistida)
Autoria	Kislley Rodrigues - Wazuh Ambassador Program
Data	Outubro de 2026

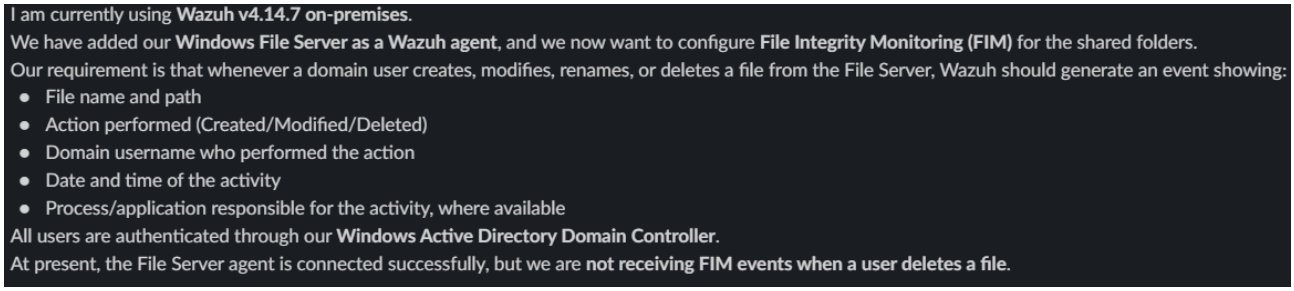
1. Sumário Executivo

O módulo **File Integrity Monitoring** (FIM) do Wazuh detecta o que mudou, quando mudou e, com whodata, quem mudou. Em produção, porém, a maioria dos incidentes de cobertura nasce de três causas, muitas vezes desconhecidas: o subsistema de auditoria do sistema operacional por baixo do FIM, as particularidades de funcionamento do wazuh-analysisd na correlação de eventos e certas condições de configuração que reduzem a cobertura sem registrar erro no log.

Este handbook consolida em um único documento o conhecimento validado em laboratório próprio sobre FIM no Wazuh 4.14.x: a arquitetura e os três modos de monitoramento, a implementação em Windows e Linux, um conjunto de seis regras de correlação para detecção comportamental de anomalias em sistemas de arquivos, o painel de investigação **FIM Alert Investigation** com análise assistida por LLM e os 15 achados técnicos validados.

As mesmas dúvidas sobre File Integrity Monitoring (FIM) aparecem recorrentemente na comunidade Wazuh, no Slack, em fóruns, no LinkedIn e em outras plataformas, especialmente entre usuários que estão implementando o FIM em produção pela primeira vez. O padrão costuma ser bastante semelhante: eventos de criação e modificação são recebidos normalmente, mas os eventos de exclusão não chegam, ou o comportamento é exatamente o oposto.

Diante desse cenário, é comum presumir que a solução esteja em uma única linha ausente no ossec.conf. Em alguns casos, essa suposição está correta. Em muitos outros, porém, a causa está em uma camada inferior à configuração do próprio FIM, seja no subsistema de auditoria do sistema operacional, seja em uma Group Policy que continua aplicada à OU do servidor, embora isso tenha sido esquecido ou passe despercebido durante a investigação.


I am currently using **Wazuh v4.14.7 on-premises.**
We have added our **Windows File Server as a Wazuh agent**, and we now want to configure **File Integrity Monitoring (FIM)** for the shared folders.
Our requirement is that whenever a domain user creates, modifies, renames, or deletes a file from the File Server, Wazuh should generate an event showing:
- File name and path
- Action performed (Created/Modified/Deleted)
- Domain username who performed the action
- Date and time of the activity
- Process/application responsible for the activity, where available

All users are authenticated through our **Windows Active Directory Domain Controller**.
At present, the File Server agent is connected successfully, but we are **not receiving FIM events when a user deletes a file**.

Figura 1 - Dúvida recorrente na comunidade Wazuh: agente em um File Server Windows (Wazuh 4.14.7) sem eventos de FIM

Esse cenário foi também uma das principais motivações para a elaboração deste handbook. A expectativa é que ele sirva como um guia de consulta rápida, claro e objetivo, ajudando a esclarecer as dúvidas mais comuns de quem ainda não possui conhecimentos avançados sobre o Wazuh, especialmente no contexto do FIM (File Integrity Monitoring).

Este material reúne e organiza os principais aspectos da arquitetura do FIM, detalhando onde cada componente da configuração está localizado, tanto no servidor Wazuh quanto nos endpoints Windows e Linux. Também aborda como as regras do Linux Audit são efetivamente construídas e persistidas, além dos modos de falha que explicam grande parte dos relatos de eventos de exclusão que não chegam ao Wazuh, conforme observado nas discussões da comunidade.

Todos os exemplos de comandos e caminhos apresentados neste material utilizam como referência o Wazuh Server 4.14.8. O conteúdo técnico foi validado em laboratório próprio, composto por um agente Windows e um agente Linux, ambos monitorados pelo mesmo Wazuh Server.

> **ATENÇÃO:** O FIM pode ficar sem cobertura sem qualquer erro registrado. Dois casos foram validados no laboratório: no Windows, a SACL do whodata não é recuperada após o restart do manager (Achado AO-7); no Linux, um caminho com `/*` em `<directories>` registra watches em arquivos individuais e deixa os arquivos novos sem cobertura realtime (Achado AO-8). Evidências e mitigações no capítulo 10.

1.1 Resultados Principais

Resultado	Detalhe	Status
Regras de correlação customizadas	100300, 100301, 100302, 100303, 100304 e a regra proxy 100310 em `fim_anomaly_rules.xml`	**VALIDADO**
Isolamento de contador no analysisd	Regra proxy 100310 separa os acumuladores de 100300 e 100301 (grupos `fim_creation` e `fim_new_entry`)	**VALIDADO**
Validação cruzada de plataformas	Windows (LABDESK) e Linux (pentlab e labnix) para as regras de criação, modificação, exclusão e escalação	**VALIDADO**
Investigação assistida por IA	FIM Alert Investigation com Claude Sonnet 4.6: latência FIM-to-verdict de 16 s medida em labnix	**VALIDADO**
Simulação pós-exploração Windows	44 alertas FIM, regra 100302 disparada, veredito `risk_score` 87 (CRITICAL) com 11 técnicas MITRE	**VALIDADO**

Resultado	Detalhe	Status
Regra 100301 em Windows	`same_field` sobre campos syscheck não acessível ao motor de correlação nos testes de 16/09/2026; limitação documentada, com cobertura de criação via 100300 no Windows	**PARCIAL**

Tabela 1 - Resumo dos resultados validados no laboratório

> **AVISO DE CALIBRAÇÃO:** Os parâmetros `frequency="3"`, `timeframe="600"` e `ignore="30"` das regras de correlação foram calibrados para laboratório. Em produção, ajuste-os contra um baseline de atividade normal por grupo de agentes antes do deploy.

1.2 Estrutura do Handbook

Capítulo	Conteúdo
2. Fundamentos do FIM	Arquitetura, onde a configuração vive, os três modos, parâmetros e a restrição de caminhos de rede
3. FIM em Windows	SACL, subcategorias de auditoria, `auditpol`, GPO, rename e monitoramento de Registro
4. FIM em Linux	auditd, regras persistentes, CDB de audit-keys, provider eBPF, inotify e checklist de diagnóstico
5. Regras de Detecção de Anomalia	Seis regras de correlação, o padrão proxy da regra 100310 e as evidências por plataforma
6. FIM Alert Investigation	Arquitetura, enriquecimento de threat intelligence, veredito do LLM e validações
7. Regras Complementares	Regras nativas que determinam a resiliência do FIM: ciclo de vida do agente, fila inotify e Registro, além da referência ao Active Response
8. Perspectiva do Adversário	Evasão do monitoramento, timestomping e padrão de eventos de ransomware
9. Compliance e MITRE ATT&CK	PCI DSS 4.0, NIST 800-53, HIPAA, GDPR, TSC e mapeamento de técnicas
10. Achados Técnicos	Os 15 achados validados, com evidência, impacto e mitigação
11. Configurações de Referência	Novidades da 4.14.8, ossec.conf endurecido, estado do servidor e conclusão
12. Referências Oficiais	Referências verificadas da documentação Wazuh e do projeto

Tabela 2 - Mapa de capítulos

1.3 Ambiente de Referência

O laboratório opera com um servidor Wazuh e agentes em plataformas distintas, o que permite validar cada cenário em ecossistemas heterogêneos. Nenhum endereço de rede do servidor ou de ambientes de produção aparece neste documento; campos de endereço usam placeholders.

Componente	Plataforma	Papel
WZ-SRV	Ubuntu 24.04 LTS, 3,9 GB de RAM, Wazuh 4.14.8	Servidor Wazuh: manager, indexer, dashboard e WPTV (Flask)
LABDESK	Windows 11 Pro (build 26200), agente 4.14.8	Agente Windows: FIM em sistema de arquivos e Registro
labnix	Ubuntu 24.04.5, kernel 6.17.0, agente 4.14.8	Agente Linux atual: realtime, whodata e auditd
pentlab	Kali Linux (kernel 6.x), agente 4.14.7	Agente Linux das validações iniciais
Inteligência	Claude Sonnet 4.6 (Anthropic), VirusTotal, AlienVault OTX, AbuseIPDB	Análise e enriquecimento do FIM Alert Investigation

Tabela 3 - Componentes do laboratório

Duas observações de rastreabilidade. Primeiro, as evidências de 4.14.7 e 4.14.8 coexistem: parte dos cenários de correlação foi executada em 4.14.7, e o pipeline atual do laboratório opera em 4.14.8. Segundo, a capacidade de laboratório é a de um servidor de 3,9 GB de RAM; os números de volume e latência aqui reportados refletem esse perfil e não substituem um dimensionamento de produção.

2. Fundamentos do FIM no Wazuh

FIM no Wazuh é implementado pelo componente syscheck, executado como o daemon `wazuh-syscheckd` em cada agente. Esse daemon mantém um baseline local dos arquivos e diretórios monitorados, capturando checksums (MD5, SHA-1, SHA-256), tamanho, permissões, proprietário, grupo e data de modificação, e compara o estado atual do sistema de arquivos contra esse baseline sempre que uma mudança é detectada ou um scan é executado.

2.1 Anatomia da Detecção

A detecção é uma comparação entre dois estados: o baseline criptográfico armazenado localmente e a realidade observada no disco. Qualquer divergência de checksum ou atributo gera um evento que o servidor decodifica e correlaciona.

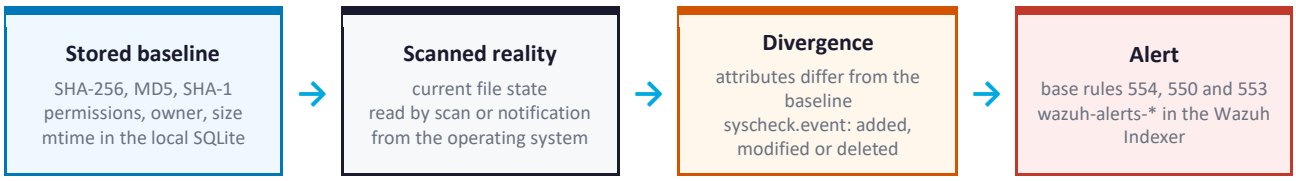


Figura 2 - Anatomia da detecção: o FIM compara o estado ao vivo contra o baseline e gera alerta ao encontrar divergência

O módulo mantém dois bancos de dados sincronizados. No endpoint, um banco local SQLite armazena o baseline em `queue/fim/db`, dentro do diretório de instalação do agente. No servidor, o daemon `wazuh-db` mantém um banco por agente, usado para servir consultas de API relacionadas a FIM e manter atualizado o inventário de arquivos exibido no dashboard. A sincronização envia apenas o que mudou: checksums e atributos divergentes do último estado conhecido.

2.2 Arquitetura Ponta a Ponta

O fluxo, conforme a documentação oficial do Wazuh, parte do sistema de arquivos monitorado, passa pelo banco local do agente (FIM DB), segue ao servidor como mensagem de evento FIM e mensagem de sincronização, é processado pelo decoder e pelo rule match no manager e termina no banco de agentes e no arquivo de alertas. No laboratório, a esse fluxo se somam o Wazuh Indexer e o painel de investigação.

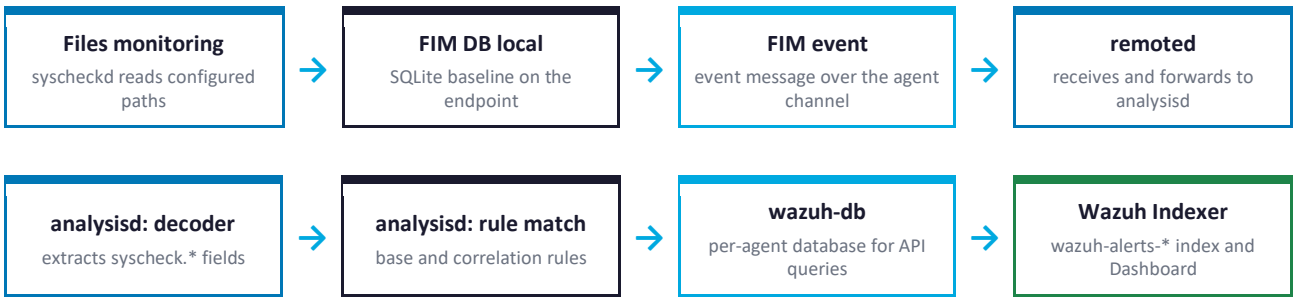


Figura 3 - Fluxo FIM de ponta a ponta: endpoint (linha superior) e servidor (linha inferior). Baseado em documentation.wazuh.com, capability File Integrity Monitoring - how it works

Decompondo em etapas: o `syscheckd` executa o scan inicial de baseline ao iniciar e popula o banco SQLite local. A partir daí, conforme o modo configurado para cada diretório, reage a mudanças por notificação do sistema operacional ou por comparação periódica contra o baseline. Quando uma divergência é encontrada, o

agente gera um evento e o envia ao servidor. O `wazuh-analysisd` decodifica e aplica as regras; se houver correspondência, o evento vira alerta, enviado ao Wazuh Indexer para armazenamento e visualização no Wazuh Dashboard.

> **Comportamento na reinicialização:** Sempre que o serviço do agente reinicia, o `syscheckd` reconstrói seu banco local e executa um novo scan completo, sincronizando o resultado com o servidor. Diretórios com realtime ou whodata sincronizam imediatamente; os demais aguardam o próximo scan agendado. Mudanças ocorridas enquanto o serviço estava parado não são reportadas retroativamente.

2.3 Onde a Configuração Vive

Grande parte dos relatos de "configurei e nada mudou" vem da edição do arquivo errado, ou de um arquivo que a configuração centralizada de grupo sobrescreve silenciosamente no próximo check-in do agente. O mapa abaixo evita esse erro.

Local	Caminho	Observação
Manager (local)	`/var/ossec/etc/ossec.conf`	Afeta apenas o host do manager
Grupo de agentes	`/var/ossec/etc/shared/<group_name>/agent.conf`	Configuração centralizada, preferível à edição por agente
Regras customizadas	`/var/ossec/etc/rules/`	Silenciar ruído e lógica de correlação
Ruleset padrão	`/var/ossec/ruleset/rules/`	Inclui 0015-ossec_rules.xml e 0365-auditd_rules.xml
CDB de audit-keys	`/var/ossec/etc/lists/audit-keys`	Mapeia chaves do Audit para categorias de evento
Agente Windows	`C:\Program Files (x86)\ossec-agent\ossec.conf`	Reinício: `Restart-Service -Name wazuh`
Agente Linux	`/var/ossec/etc/ossec.conf`	Reinício: `systemctl restart wazuh-agent`
Agente macOS	`/Library/Ossec/etc/ossec.conf`	Reinício: `/Library/Ossec/bin/wazuh-control restart`

Tabela 4 - Localização dos arquivos de configuração FIM por componente

As três plataformas de agente são editadas dentro do bloco `<syscheck>` e exigem reinício do serviço para aplicar mudanças. O `syscheckd` só reconhece novos diretórios, e só autoconfigura SACL ou regras de auditoria para eles, após um reinício, e não em um reload a quente.

2.4 Os Três Modos de Monitoramento

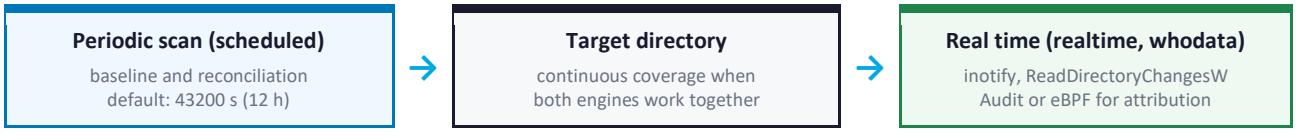


Figura 4 - Motor dual de monitoramento: varredura periódica para baseline e tempo real para captura imediata, em paralelo

Modo	Mecanismo	Atribuição de usuário	Overhead	Quando usar
scheduled	Scan completo a cada `frequency` (padrão 43200 s) ou em `scan_time` e `scan_day`	Não	Menor	Caminhos estáticos e de baixa movimentação
realtime	`inotify` (Linux) e `ReadDirectoryChangesW` (Windows); apenas diretórios	Não	Médio	Detecção imediata sem exigência de "quem"
whodata	Subsistema de auditoria: SACL no Windows; auditd ou eBPF no Linux	Sim (usuário e processo)	Maior	Compliance e resposta a incidentes

Tabela 5 - Comparação dos modos de monitoramento do syscheck

Scan agendado

É o modo padrão e de menor overhead. Um arquivo criado, modificado e excluído inteiramente entre dois scans não produz nenhum evento, porque os dois estados comparados nunca capturaram a mudança transitória. É o modo em que muitos ambientes acabam caindo sem perceber, ao adicionar um diretório sem os atributos `realtime` ou `whodata`.

Tempo real

Usa a API nativa de notificação do sistema operacional para detectar criação, modificação e exclusão no instante em que acontecem. É habilitado por diretório com `realtime="yes"` e não funciona em arquivos individuais. Informa o que mudou e quando, mas não quem nem qual processo, porque a API de notificação não carrega essa informação.

Who-data

Adiciona atribuição de usuário e processo sobre a detecção em tempo real, usando o subsistema de auditoria do sistema operacional. É exigido sempre que um requisito inclui "**quem fez a mudança**", o que cobre a maioria dos deployments orientados a compliance e os casos de resposta a incidentes. Se o whodata não inicializar, o módulo faz fallback automático para realtime, perdendo a atribuição silenciosamente enquanto ainda detecta a mudança.

```
<syscheck>
  <directories check_all="yes" whodata="yes"
report_changes="yes">/caminho/monitorado</directories>
</syscheck>
```

2.5 Parâmetros de Configuração

Parâmetro	Função	Observação operacional
`check_all`	Verifica todos os atributos: tamanho, permissões, dono, grupo, checksums e mtime	Base recomendada para qualquer diretório monitorado com seriedade
`whodata`	Atribuição de usuário e processo	Exige auditoria do SO disponível (capítulos 3 e 4)
`realtime`	Detecção imediata via API do SO, sem atribuição	Redundante junto de `whodata`, que já implica tempo real
`report_changes`	Diff de conteúdo textual, guardando cópia do arquivo	Útil em configuração e scripts; em binários consome espaço em `queue/diff` sem valor

Parâmetro	Função	Observação operacional
`restrict`	Filtro por expressão regular no nome do arquivo	Principal ferramenta para reduzir ruído em diretórios grandes
`recursion_level`	Profundidade de subdiretórios no scan	Padrão 256, máximo 320; 0 monitora só o diretório informado
`frequency`	Intervalo do scan agendado, em segundos (padrão 43200)	Não afeta a detecção de diretórios realtime ou whodata
`scan_time`, `scan_day`	Fixam o scan completo em horário e dia específicos	Úteis para janelas de manutenção

Tabela 6 - Parâmetros do bloco directories e do syscheck

2.6 A Restrição de Caminhos de Rede

A partir da versão 4.13.0, mantida na 4.14.x, o FIM no Windows não suporta caminhos de rede UNC, como `\\servidor\compartilhamento\pasta`, nem drives mapeados, como uma letra `Z:\pasta` apontando para um host remoto. Qualquer caminho desse tipo declarado na configuração é ignorado e não gera eventos, em nenhum modo. Segundo as notas de versão da Wazuh 4.13.0 (#30060), a restrição corrige a exposição de NetNTLMv2 causada pelo monitoramento de caminhos UNC e drives mapeados.

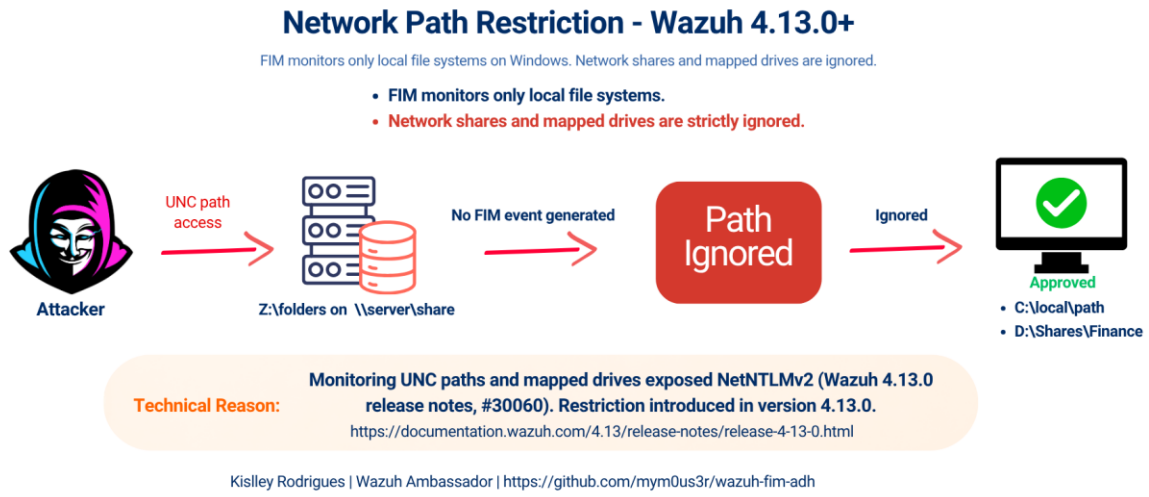


Figura 5 - Restrição de caminhos de rede no Wazuh 4.13.0 e posteriores: pastas compartilhadas devem ser monitoradas pelo caminho local no servidor

2.7 Sincronização Delta

O bloco `synchronization` dentro de `syscheck` não controla quando os alertas de FIM são enviados; controla quando o banco local do agente é reconciliado com o banco do servidor. São mecanismos distintos: os alertas viajam em tempo real pelo canal de eventos, enquanto a sincronização envia apenas deltas.



Figura 6 - Sincronização delta: dados não alterados nunca cruzam a rede

```
<synchronization>
  <enabled>yes</enabled>
  <interval>5m</interval>
  <max_eps>10</max_eps>
</synchronization>
```

O intervalo de 5 minutos é o tempo inicial entre sincronizações; se uma falhar, o intervalo dobra até o `max_interval` (padrão 1h). O `max_eps` de 10 limita a taxa de mensagens de sincronização para não saturar o `analysisd` na reconexão de um agente que ficou offline por muito tempo.

> **IMPLICAÇÃO PARA INVESTIGAÇÃO:** Se um agente ficou offline durante um ataque, a reconexão não reenvia os eventos do período. O servidor calcula o delta entre o estado anterior à desconexão e o estado atual. Um arquivo criado e depois excluído enquanto o agente estava offline resulta em delta zero e nenhum alerta. Uma lacuna no histórico FIM nunca deve ser lida como ausência de atividade: o timestamp de reconexão do agente é o primeiro dado a verificar antes de qualquer análise de timeline.

2.8 Boas Práticas Gerais

- **Revise frequency e use scan_time.** Em caminhos monitorados apenas por agendamento, um valor menor reduz a janela de detecção e aumenta o I/O proporcionalmente; `scan_time` e `scan_day` deslocam o scan para uma janela de manutenção.
- **Use restrict e ignore com rigor em diretórios ruidosos.** Um diretório monitorado para persistência de malware raramente precisa reportar cada arquivo temporário.
- **Reserve report_changes para onde o diff tem valor,** evitando consumo desnecessário de disco em diretórios binários.
- **Não existe evento nativo de "renomeado"** em nenhuma plataforma. Um rename é reportado como exclusão do caminho antigo seguida de criação do caminho novo; correlacioná-los como ação única exige regra customizada.

3. FIM em Windows

Este capítulo cobre a implementação de FIM em endpoints Windows monitorados por um Wazuh Server 4.14.x: a autoconfiguração de política de auditoria feita pelo agente, os pontos de falha mais comuns e a validação correta em ambiente domain joined.

3.1 Exemplos de Diretórios Monitorados

Além do conjunto padrão que o Wazuh traz (`%WINDIR%`, `System32`, `SysNative` e a pasta de Startup), os exemplos abaixo ilustram casos de uso distintos em um endpoint Windows típico.

```
<syscheck>
  <directories check_all="yes" whodata="yes"
report_changes="yes">D:\Shares\Finance</directories>
  <directories check_all="yes" whodata="yes">C:\Program Files</directories>
  <directories check_all="yes" whodata="yes">C:\Program Files (x86)</directories>
  <directories check_all="yes" whodata="yes"
restrict="\.exe$|\.dll$">C:\Users\Public</directories>
  <windows_registry
arch="both">HKEY_LOCAL_MACHINE\Software\Microsoft\Windows\CurrentVersion\Run</windows_regist
ry>
</syscheck>
```

- **D:\Shares\Finance** é o cenário de File Server do capítulo: `whodata` e `report_changes` entregam usuário, processo e diff de conteúdo.
- **Program Files e Program Files (x86)** são alvos comuns de persistência. Vale `check_all` e `whodata`, sem `report_changes`, pois o diff binário não tem valor de leitura.
- **C:\Users\Public com restrict** limita o alerta a executáveis e DLLs depositados em um local clássico de staging para movimentação lateral, sem ruído de arquivos temporários.
- **A chave Run do Registro** entra porque o FIM cobre nativamente o Registro do Windows, e esse é um dos locais de persistência de maior valor.

3.2 Autoconfiguração de SACL e Política de Auditoria

Quando `whodata="yes"` é declarado para um diretório e o serviço do agente reinicia, o Wazuh não apenas passa a escutar: ele modifica ativamente o sistema operacional. Aplica uma System Access Control List (SACL) ao diretório monitorado e habilita as subcategorias de auditoria necessárias para gerar as entradas correspondentes no log de segurança. Em condições normais, em uma máquina standalone, isso é automático e dispensa `auditpol` ou `secpol.msc`.

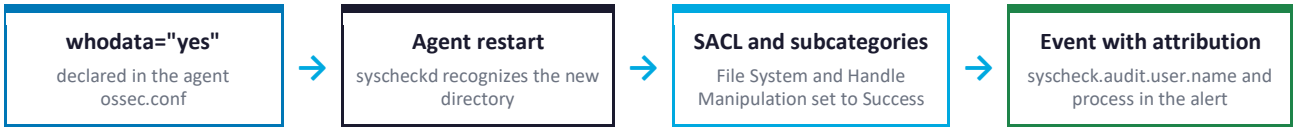


Figura 7 - Cadeia de ativação do whodata no Windows: da declaração à atribuição de usuário

Duas subcategorias da Advanced Audit Policy, ambas sob Acesso a Objetos, são exigidas, e a documentação do Wazuh é explícita quanto à necessidade das duas.

Subcategoria	GUID	O que cobre
Sistema de Arquivos (File System)	`{0CCE921D-69AE-11D9-BED3-505054503030}`	Evento de acesso a objeto quando um handle é concedido, com base na SACL do objeto
Manipulação de Identificador (Handle Manipulation)	`{0CCE9223-69AE-11D9-BED3-505054503030}`	Fechamento ou duplicação de handle; carrega a informação que atribui a exclusão de arquivo

Tabela 7 - Subcategorias de auditoria exigidas pelo whodata no Windows

> **SINTOMA MAIS COMUM:** Com apenas Sistema de Arquivos habilitada, o whodata captura criação e modificação de forma confiável e falha em atribuir, ou perde por completo, a exclusão. Exclusões ausentes com criações e modificações normais é o sintoma mais relatado na comunidade, e Manipulação de Identificador não configurada é a causa raiz mais comum.

3.3 Validação com auditpol

Validar o estado efetivo dessas subcategorias nunca deve ser feito lendo o console `secpol.msc`. O nó de Configuração de Diretiva de Auditoria Avançada reflete o template de GPO local em disco, não necessariamente a política efetiva em execução. Quando uma subcategoria é definida pela API de política de auditoria do Windows, exatamente o mecanismo da autoconfiguração do Wazuh, a mudança vale de imediato na política efetiva, enquanto o `secpol.msc` pode continuar exibindo Não Configurado. A forma confiável de confirmar o estado efetivo é o `auditpol`, consultando por GUID para evitar falhas em instalações de Windows fora do inglês.

```
auditpol /get /subcategory:"{0CCE9223-69AE-11D9-BED3-505054503030}","{0CCE921D-69AE-11D9-BED3-505054503030}"
```

O primeiro GUID corresponde a Manipulação de Identificador e o segundo a Sistema de Arquivos. Ambos precisam retornar Êxito. No PowerShell, o GUID precisa estar entre aspas, porque o PowerShell interpreta as chaves como abertura de script block e o comando falha com um erro genérico de parâmetro incorreto, sem relação com a política de auditoria.

> **Idioma do sistema operacional:** Os nomes de subcategoria variam conforme o idioma. No Windows em português do Brasil aparecem como "Manipulação de Identificador" e "Sistema de Arquivos"; em inglês, como "Handle Manipulation" e "File System". Se o `auditpol` falhar com o nome, use sempre a consulta por GUID.

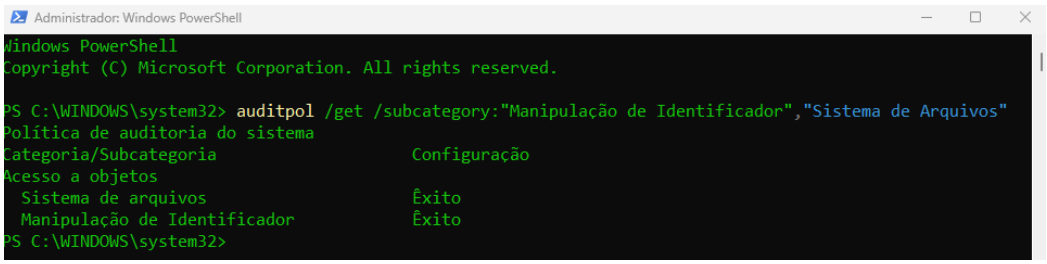


Figura 8 - Evidência real: auditpol confirmando Êxito em Sistema de Arquivos e Manipulação de Identificador no LABDESK

Nota de laboratório: a captura do estado "antes" da diretiva whodata não foi realizada no LABDESK para não desfazer a configuração já validada. Em produção, capture o estado inicial antes de qualquer mudança como baseline documentado para auditoria e rollback.

3.4 Política Local versus GPO em Ambiente Domain Joined

Em um servidor domain joined, a autoconfiguração local não é o fim da história. Se uma GPO de Advanced Audit Policy estiver vinculada à unidade organizacional do File Server e definir explicitamente Sistema de Arquivos ou Manipulação de Identificador, essa GPO é reaplicada a cada ciclo de atualização de política de grupo, por padrão a cada 90 a 120 minutos. Se definir essas subcategorias com valor diferente do que o Wazuh precisa, ou simplesmente as resetar, cada atualização reverte silenciosamente a autoconfiguração, e a atribuição de exclusão volta a falhar horas depois de ter sido confirmada, sem erro visível na stack Wazuh. Se a GPO deixar as duas subcategorias como Não Configurado, ela não interfere.

Para determinar qual cenário se aplica, execute `gpresult /h relatorio.html` no servidor e inspecione a seção Acesso a Objeto em busca das duas subcategorias, observando qual GPO, se houver, aparece como política vencedora. Se uma GPO conflitante vencer, a correção durável é atualizá-la para definir explicitamente as duas subcategorias como Êxito na OU relevante.

> **Limite da validação:** A evidência de `gpresult` exige um File Server domain joined. O LABDESK opera em workgroup, sem vínculo a Active Directory. O comportamento descrito acima está documentado com base na documentação oficial do Wazuh e em registros da comunidade, e não foi reproduzido neste laboratório.

3.5 windows_audit_interval em Profundidade

O parâmetro `windows_audit_interval` consta no `ossec.conf` padrão e é ignorado pela maioria dos deployments porque o nome não sugere sua função. Ele não controla a frequência do scan de FIM: controla a frequência com que o Wazuh verifica se a SACL do diretório monitorado e as subcategorias de auditoria ainda estão corretamente configuradas.

```
<windows_audit_interval>60</windows_audit_interval>
```

A documentação oficial descreve dois mecanismos de detecção de reversão externa da auditoria: o monitoramento do EventID 4719, gerado quando uma política de auditoria é modificada, e o próprio `windows_audit_interval`, que verifica periodicamente as políticas de auditoria e as SACLs dos diretórios monitorados. Quando qualquer um dos mecanismos detecta divergência, o FIM move todos os diretórios whodata para realtime, mantendo a detecção sem atribuição. A documentação oficial não descreve reaplicação automática da política.



Figura 9 - Defesa em camadas da política de auditoria: detecção da reversão e fallback para realtime

> **LIMITE VALIDADO EM LABORATÓRIO:** Após o restart do manager, o whodata no LABDESK ficou cerca de 25 minutos sem gerar eventos syscheck, até um `Restart-Service WazuhSvc` manual reaplicar a SACL. O `windows_audit_interval` apenas verifica a política e as SACLs e, em caso de divergência, move os diretórios para realtime; não reaplica a SACL. Detalhes no Achado AO-7 (capítulo 10).

O valor padrão é 300 segundos, que pode ser inadequado onde o refresh de GPO é frequente ou o risco de tamper intencional é alto. O valor 60 usado no laboratório pode ser mantido em ambientes de alta sensibilidade, ciente de que cada verificação consulta a API do Windows, com custo de CPU mínimo mas não nulo em escala.

3.6 Rename como Exclusão mais Criação

Não existe tipo de evento nativo "renomeado". Um rename é reportado como duas operações: exclusão do caminho antigo (rule ID 553) e criação do caminho novo (rule ID 554), com o mesmo timestamp e a mesma atribuição de usuário quando o whodata está ativo. Se um requisito exige que renomes apareçam como ação única, a correlação precisa ser uma regra customizada que case um evento 553 e um 554 com o mesmo usuário e uma janela de tempo curta.

3.7 Monitoramento do Registro do Windows

O FIM do Wazuh também cobre o Registro do Windows por decoders e regras próprios, mapeados ao MITRE T1112 (Modify Registry). Um malware que cria um executável em diretório monitorado e registra uma chave Run para persistir gera dois alertas correlacionáveis: um 554 (arquivo adicionado) e um 752 (valor de registro adicionado).

Rule ID	Alvo	Evento	MITRE
594	Chave	Registry Key Integrity Checksum Changed	T1565.001, T1112
597	Chave	Registry Key Entry Deleted	T1070.004, T1485, T1112
598	Chave	Registry Key Entry Added	T1112
750	Valor	Registry Value Integrity Checksum Changed	T1565.001, T1112
751	Valor	Registry Value Entry Deleted	T1070.004, T1485, T1112
752	Valor	Registry Value Entry Added	T1112

Tabela 8 - Regras nativas de FIM de Registro do Windows

```
<windows_registry
arch="both">HKEY_LOCAL_MACHINE\Software\Microsoft\Windows\CurrentVersion\Run</windows_regist
ry>
<windows_registry
arch="both">HKEY_LOCAL_MACHINE\Software\Microsoft\Windows\CurrentVersion\RunOnce</windows_re
gistry>
<windows_registry
arch="both">HKEY_LOCAL_MACHINE\System\CurrentControlSet\Services</windows_registry>
<windows_registry arch="both">HKEY_LOCAL_MACHINE\Software\Microsoft\Windows
NT\CurrentVersion\Winlogon</windows_registry>
```

O atributo `arch="both"` monitora as visões de 32 e 64 bits do Registro, relevante porque malware de 32 bits em sistemas de 64 bits pode gravar em locais virtualizados pelo WoW64 que não aparecem na visão padrão de 64 bits.

3.8 Evidência de Laboratório: Ciclo Completo

Para validar o ciclo, crie, modifique e exclua um arquivo de teste no diretório declarado com whodata e capture o resultado no Discover, filtrando por `agent.name` e `rule.group:syscheck`. Com configuração e auditoria corretas, o resultado são os três eventos, added, modified e deleted, com rule.id 554, 550 e 553, path correto e `syscheck.audit.user.name` preenchido nos três.

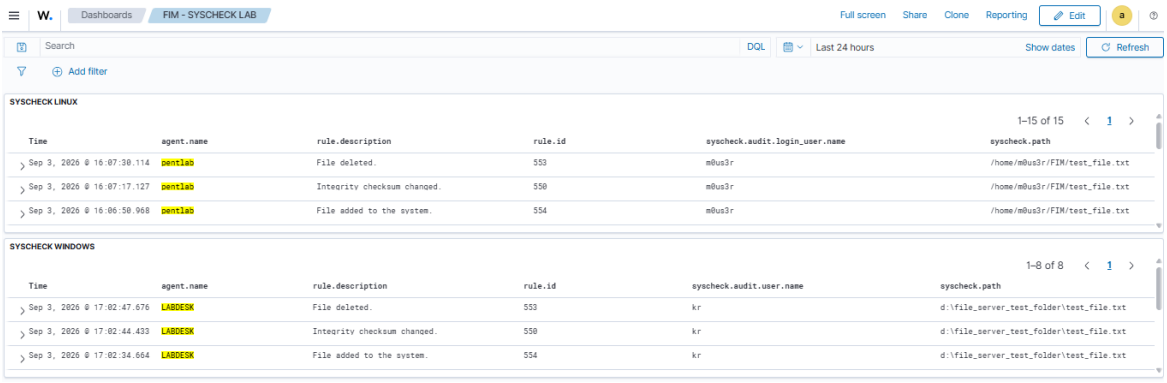


Figura 10 - Evidência real: painel FIM - SYSCHECK LAB comparando eventos de syscheck no agente Linux (pentlab) e no agente Windows (LABDESK), com atribuição de usuário em ambos

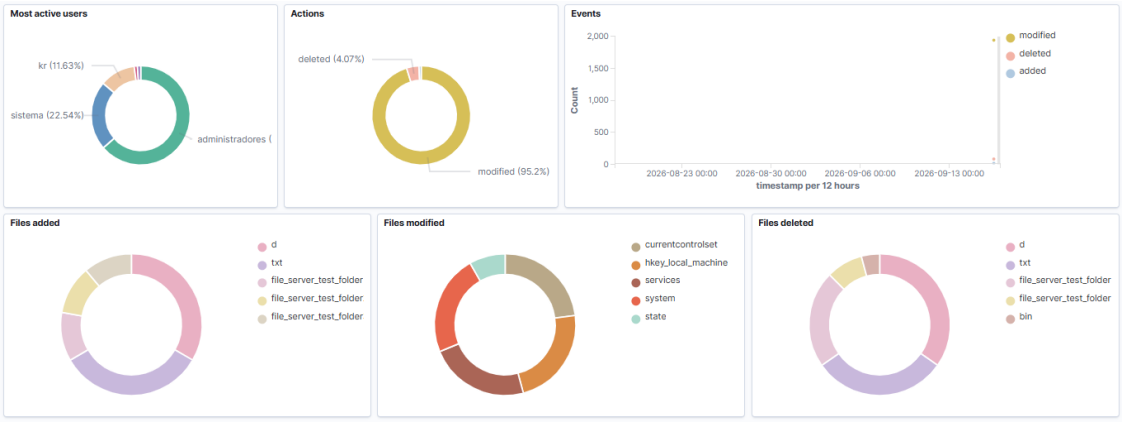


Figura 11 - Evidência real: dashboard de FIM do agente LABDESK (últimos 30 dias) com distribuição de eventos por tipo: modified, deleted e added

4. FIM em Linux

Este capítulo cobre FIM em endpoints Linux, com foco no mecanismo equivalente ao whodata do Windows, o subsistema Linux Audit (auditd), e na alternativa baseada em eBPF disponível desde a versão 4.12.0.

4.1 Exemplos de Diretórios Monitorados

```
<syscheck>
  <directories check_all="yes"
whodata="yes">/etc,/usr/bin,/usr/sbin,/bin,/sbin,/boot,/root</directories>
  <directories check_all="yes" whodata="yes"
report_changes="yes">/var/www/html</directories>
  <directories check_all="yes"
whodata="yes">/etc/passwd,/etc/shadow,/etc/sudoers</directories>
</syscheck>
```

A primeira linha cobre o baseline clássico de binários de sistema e arquivos de configuração recomendado para qualquer host Linux endurecido. A segunda mira uma raiz web com diffing de conteúdo, útil para capturar webshells e defacement. A terceira isola os três arquivos mais visados para escalada de privilégio e persistência, justificando uma linha própria para que uma regra customizada alerte sobre eles em severidade maior que uma mudança genérica em `/etc`.

4.2 Instalação e Configuração do auditd

Quando `whodata="yes"` é declarado para um diretório, o Wazuh cria automaticamente as regras de auditoria correspondentes no kernel e, por padrão, reinicia o auditd para aplicá-las. Se o auditd não estiver instalado, em Debian ou Ubuntu:

```
apt install -y auditd
```

Em RHEL, CentOS ou AlmaLinux, use `yum` ou `dnf install auditd`, conforme a versão. Para o Audit versão 3.1.1 ou superior, é necessário instalar também o pacote `audispd-plugins`, que fornece o plugin `audisp-af_unix`, responsável pela entrega de eventos ao socket do Wazuh, e reiniciar o auditd para carregar o plugin.

```
yum install audispd-plugins
systemctl restart auditd
```

Duas configurações precisam existir no endpoint. Primeiro, o agente lê o próprio log de auditoria:

```
<localfile>
  <log_format>audit</log_format>
  <location>/var/log/audit/audit.log</location>
</localfile>
```

Segundo, o comportamento de reinício do auditd é controlado por `restart_audit` dentro do bloco `whodata`:

```
<whodata>
  <provider>audit</provider>
  <restart_audit>yes</restart_audit>
  <audit_key>auditkey1,auditkey2</audit_key>
  <startup_healthcheck>yes</startup_healthcheck>
  <queue_size>100000</queue_size>
</whodata>
```

RESTART_AUDIT=NO: Definir `restart_audit` como `no` significa que novas regras de whodata não são aplicadas automaticamente: o equivalente Linux de um Windows onde a SACL nunca chega a ser aplicada. O whodata fica declarado no `ossec.conf`, mas o subsistema de auditoria por baixo nunca é configurado para sustentá-lo.

4.3 Regras de Auditoria Manuais e Persistência

Para monitorar caminhos fora do FIM cujo acesso ainda precise ser rastreado, a sintaxe do `auditctl` é:

```
auditctl -w /etc -p wa -k wazuh_fim
auditctl -w /usr/bin -p wa -k wazuh_fim
auditctl -w /usr/sbin -p wa -k wazuh_fim
auditctl -w /root -p wa -k wazuh_fim
```

A flag `-w` nomeia o caminho, `-p wa` restringe a regra a escrita e mudança de atributo (a combinação relevante para FIM) e `-k` anexa uma chave pesquisável a cada evento gerado, como o Wazuh faz nas regras que cria sozinho. Regras criadas via `auditctl` vivem apenas na memória do kernel e não sobrevivem a um reboot. Para persisti-las, escreva as mesmas regras em `/etc/audit/rules.d/`, por exemplo `/etc/audit/rules.d/wazuh-fim.rules`, e carregue com `augenrules --load`. Um servidor que perde a atribuição de whodata após um reboot de manutenção, com a configuração do Wazuh intacta, quase certamente perdeu exatamente isso: regras criadas e nunca persistidas.

Filtros de usuário: -F auid

O filtro mais relevante para FIM orientado a compliance e investigação é por `auid` (audit user ID), o UID da sessão de login, definido no login e herdado por todos os processos filhos.

Objetivo	Regra	Efeito
Apenas ações de root	`auditctl -w /etc -p wa -F auid=0 -k wazuh_fim_root`	Captura eventos de sessões de root; rastreia ações administrativas suspeitas sem ruído de sistema
Usuários regulares	`auditctl -w /etc -p wa -F auid>=1000 -F auid!=4294967295 -k wazuh_fim_users`	Exclui o loginuid inválido (4294967295, equivalente a -1) de daemons sem sessão interativa
Todas as origens	`auditctl -w /etc -p wa -k wazuh_fim_all`	Sem filtro de auid: inclui processos de sistema

Tabela 9 - Filtros de auid em regras de auditoria para FIM

> **Nota de laboratório:** Com `whodata="yes"`, o Wazuh gera as regras de auditoria sem filtro `-F auid`. Processos do próprio agente e de sistema (journald, cron, apt) que escrevem em `/etc` aparecem como eventos whodata. Em ambientes ruidosos, adicione manualmente `-F auid>=1000` em `/etc/audit/rules.d/` após a autoconfiguração. O Wazuh carrega suas regras wazuh_fim no kernel quando o agente inicia; elas são independentes dos arquivos em /etc/audit/rules.d/ (labnix: 4 regras persistidas em /etc/audit/rules.d/net_connect.rules mais 6 regras wazuh_fim geradas pelo agente, uma para cada diretório whodata: /etc, /usr/bin, /usr/sbin, /bin, /sbin e /boot).

4.4 CDB de audit-keys e a Regra 80705

Eventos brutos de Audit são ruidosos. O Wazuh usa o argumento de chave para classificar o que gerou cada entrada, por meio de uma CDB list no servidor, `/var/ossec/etc/lists/audit-keys`.

```
audit-wazuh-w:write
audit-wazuh-r:read
audit-wazuh-a:attribute
audit-wazuh-x:execute
audit-wazuh-c:command
```

Uma chave customizada é adicionada incluindo a linha com a chave usada em `-k` e a categoria desejada:

```
echo "wazuh_fim:write" >> /var/ossec/etc/lists/audit-keys
systemctl restart wazuh-manager
```

O ruleset padrão para eventos do Audit está em `/var/ossec/ruleset/rules/0365-auditd_rules.xml`. Alertas de Linux Audit, diferentemente de alertas de FIM puro, aparecem no módulo Threat Hunting do Wazuh Dashboard e não no módulo File Integrity Monitoring, ponto comum de confusão.

O rule ID **80705**, "Auditd: Configuration changed", é uma ferramenta de diagnóstico: dispara sempre que a configuração do subsistema de auditoria muda, incluindo o instante em que o Wazuh aplica sua autoconfiguração ao reiniciar o agente. Filtrar o Threat Hunting por esse ID no horário do restart confirma que a autoconfiguração tocou o auditd, o equivalente Linux do EventID 4719 do Windows.

timestamp	agent.name	rule.description	rule.level	rule.id
Sep 3, 2026 @ 16:07:30.114	pentlab	File deleted.	7	553
Sep 3, 2026 @ 16:07:17.127	pentlab	Integrity checksum changed.	7	550
Sep 3, 2026 @ 16:06:50.968	pentlab	File added to the system.	5	554
Sep 3, 2026 @ 16:05:18.046	pentlab	Auditd: Configuration changed.	3	80705
Sep 3, 2026 @ 16:05:17.998	pentlab	Auditd: Configuration changed.	3	80705

Figura 12 - Evidência real: Threat Hunting no pentlab exibindo a regra 80705 (Auditd: Configuration changed) no momento da autoconfiguração, junto dos eventos FIM 554, 550 e 553

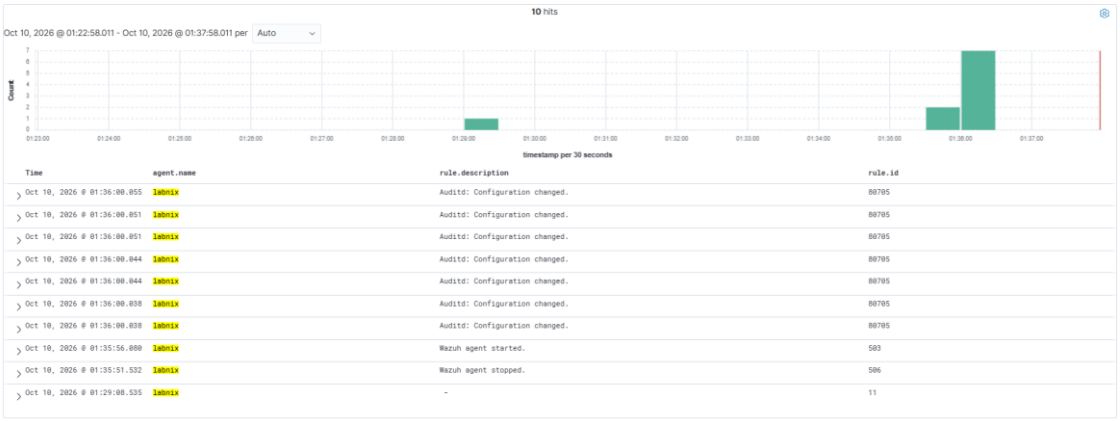


Figura 13 - Discover do labnix, 10/10/2026: sete eventos da regra 80705 (Auditd: Configuration changed) às 01:36:00, logo após a parada (regra 506, 01:35:51) e o início (regra 503, 01:35:56) do agente

4.5 Provider eBPF

O Extended Berkeley Packet Filter (eBPF) executa programas de forma segura no espaço de kernel, sem alterar o código-fonte do kernel nem carregar módulos. O modo eBPF do whodata, disponível desde a 4.12.0, extrai eventos de FIM diretamente do kernel por uma estrutura `ring_buffer`, que os transfere ao espaço de usuário onde o módulo FIM os processa. Os três eventos rastreados mapeiam as operações de VFS do kernel:

Hook do kernel	Evento FIM
`vfs_open`	Criação de arquivo (file added)
`security_inode_setattr`	Modificação de arquivo (file modified)
`vfs_unlink`	Remoção de arquivo (file deleted)

Tabela 10 - Hooks de kernel rastreados pelo provider eBPF

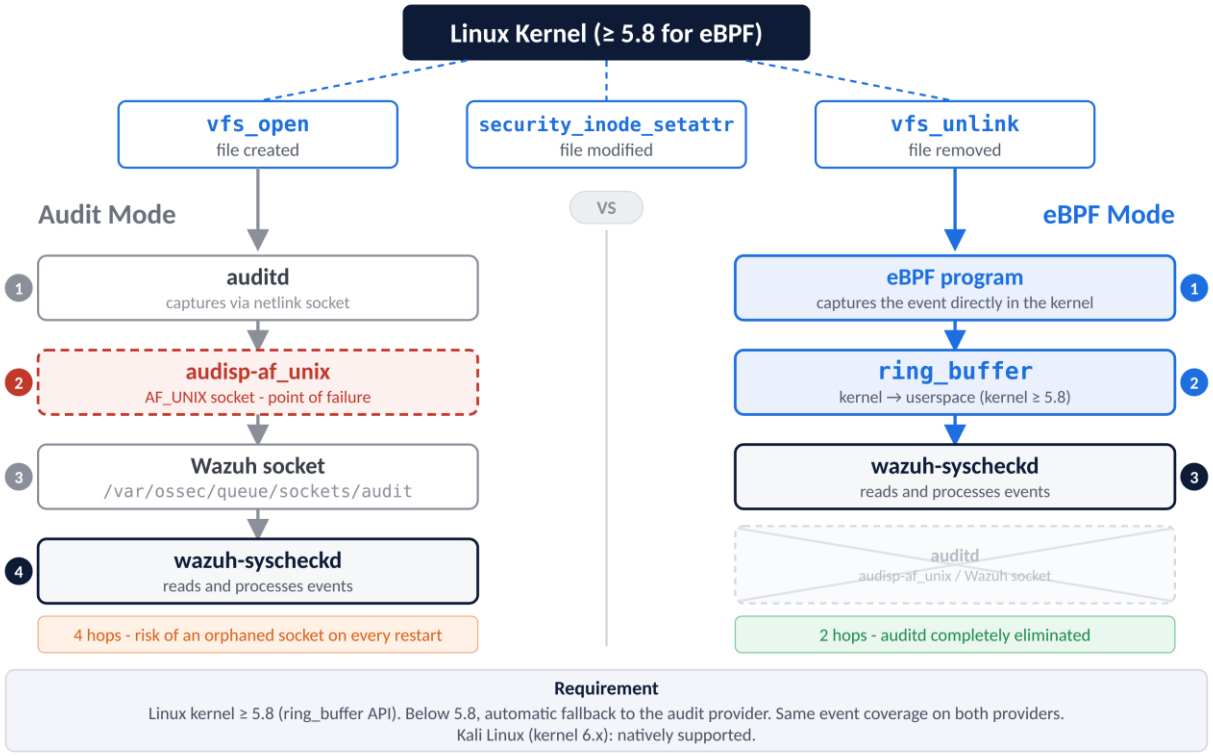


Figura 14 - Fluxo do modo audit (4 etapas) contra o modo eBPF (3 etapas): o eBPF elimina a dependência do auditd e a classe de falhas associada

A principal diferença é a eliminação do auditd, o que remove a classe de falhas associada, incluindo o problema de socket órfão da seção 4.6, sem custo de cobertura de eventos. A configuração é idêntica à do audit, exceto pelo `provider`:

```
<syscheck>
  <directories whodata="yes">/home/user/FIM</directories>
  <whodata>
    <provider>ebpf</provider>
    <queue_size>50000</queue_size>
  </whodata>
</syscheck>
```

- **Requisito de kernel:** 5.8 ou superior, pois o `ring_buffer` foi introduzido nessa versão. Em kernels anteriores, o Wazuh faz fallback automático para o provider audit.
- **queue_size:** o eBPF detecta eventos na velocidade do kernel, sem o gargalo do auditd. Rajadas, como exclusão massiva ou backup, podem superar a capacidade padrão (16384). Valores de 50000 ou mais previnem perda, ao custo de mais memória no processo do agente.
- **Providers exclusivos:** `audit` e `ebpf` são mutuamente exclusivos por execução; se ambos forem declarados, apenas o último tem efeito.
- **Fallback para audit:** um deployment configurado como `ebpf` roda como `audit` quando o health check do eBPF falha, por exemplo em kernels sem BPF ring buffer e BTF. O agente registra (6959) e nenhum alerta é gerado, o que pode ocorrer em frotas com kernels mistos.

```
# Check the kernel version before setting the provider:
uname -r
# Log line for the started whodata engine (the string is "Whodata", without a hyphen):
wazuh-syscheckd: INFO: (6019): File integrity monitoring real-time Whodata engine started.
```

4.6 Falha Comum: Socket Órfão do audisp-af_unix

Um cenário reproduzido em laboratório merece seção própria porque o sintoma não aponta para a causa. Com o auditd ativo e sem erro aparente no `systemctl status`, o `journalctl` do serviço mostra o plugin `audisp-af_unix` em loop de falha:

```
audisp-af_unix[NNN]: Couldn't bind af_unix socket (Address already in use)
audisp-af_unix[NNN]: audisp-af_unix plugin exiting due to errors setting up socket
auditd[NNN]: plugin /sbin/audisp-af_unix terminated unexpectedly
auditd[NNN]: plugin /sbin/audisp-af_unix has exceeded max_restarts
```

Esse plugin entrega os eventos de auditoria ao socket que o Wazuh consome em `/var/ossec/queue/sockets/audit`. Com ele morto, o whodata não funciona, mesmo com as regras carregadas no kernel e o `af_wazuh.conf` sintaticamente correto. Em socket AF_UNIX, "Address already in use" no bind normalmente não indica outro processo vivo: indica que o arquivo de socket já existe em disco, remanescente de uma execução que não encerrou de forma limpa.

```
ls -la /var/ossec/queue/sockets/audit
# If the file date is earlier than the last service restart:
systemctl stop auditd
rm -f /var/ossec/queue/sockets/audit
systemctl start auditd
```

O `journalctl` após a correção mostra o plugin inicializando sem erro, com o dispatcher reportando um plugin ativo e sem menção a bind ou restart excedido.

4.7 Validação e Evidência de Laboratório

O health check de inicialização existe porque o whodata no Linux depende de estado externo (regras carregadas e socket acessível) que pode divergir silenciosamente. Desabilitar `startup_healthcheck` remove essa proteção e pode deixar o módulo em estado degradado sem aviso. Uma assinatura de falha comum no `ossec.log`: o `syscheckd` reporta ausência de socket em `queue/sockets/audit`, tenta reiniciar o auditd, falha ao reconectar e faz fallback para realtime sem alerta, produzindo eventos sem atribuição de usuário ou processo, como no Windows.

```
# Confirm that the Wazuh rules were loaded in the kernel:
auditctl -l | grep wazuh_fim
-w /home/user/FIM -p wa -k wazuh_fim
# Detect loss of audit events and enlarge the buffer:
auditctl -s | grep lost
auditctl -b 16384
```

Um resultado vazio de `auditctl -l` com whodata declarado e o agente reiniciado aponta para falha no caminho do `restart_audit`; confira as mensagens de health check do `ossec.log` antes de assumir configuração errada. Quando a fila interna do dispatcher enche, alguns eventos de auditoria se perdem, e o próximo scan agendado gera os alertas faltantes sem a informação de whodata, o que parece, no dashboard, idêntico a uma má configuração.

> **Detalhe de troubleshooting:** Buscar por "who-data", com hífen, no `ossec.log` não retorna linhas. A string gravada é "Whodata", sem hífen. Um `grep` malcalibrado leva à conclusão errada de que o motor não subiu.

O scan agendado e o whodata não são mutuamente exclusivos. O trecho real do `ossec.log` abaixo mostra as duas atividades lado a lado, independentes: o scan de baseline segue como reconciliação periódica mesmo com a detecção em tempo real ativa.

```
2026/09/03 16:06:04 wazuh-syscheckd: INFO: FIM sync module started.
2026/09/03 16:06:04 wazuh-syscheckd: INFO: (6019): File integrity monitoring real-time
Whodata engine started.
2026/09/03 16:07:05 wazuh-syscheckd: INFO: (6008): File integrity monitoring scan started.
2026/09/03 16:08:00 wazuh-syscheckd: INFO: (6009): File integrity monitoring scan ended.
```

No Linux, a atribuição de usuário aparece em `syscheck.audit.login_user.name`, nome de campo diferente do usado no Windows (`syscheck.audit.user.name`).

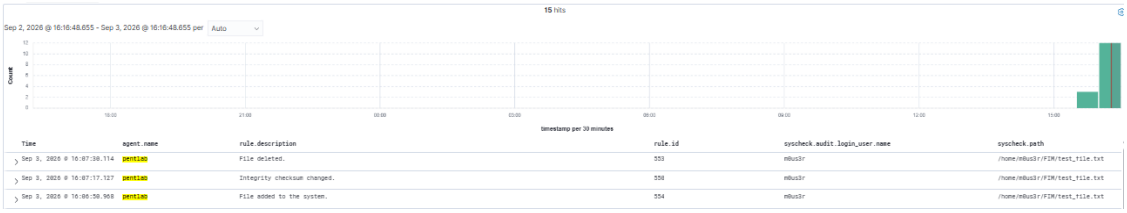


Figura 15 - Evidência real: Discover do agente pentlab, /home/m0us3r/FIM/test_file.txt, com os três eventos do ciclo e atribuição em syscheck.audit.login_user.name

4.8 Limite de inotify Watches

Este é um problema de produção do FIM em Linux que passa despercebido com facilidade, porque a falha é silenciosa. O kernel limita as instâncias de inotify watch por usuário, e o limite afeta diretamente os modos realtime e whodata. Cada diretório com `realtime` ou `whodata` consome uma instância; o padrão na maioria das distribuições é 8192. Ao atingi-lo, o kernel rejeita novos watches em silêncio: o agente aparece ativo, o scan agendado funciona, mas os eventos em tempo real param de chegar para os diretórios que excederam o limite.

```
cat /proc/sys/fs/inotify/max_user_watches
cat /proc/$(pidof wazuh-agentd)/fdinfo/* 2>/dev/null | grep -c inotify
echo 'fs.inotify.max_user_watches=524288' >> /etc/sysctl.conf
sysctl -p
sysctl fs.inotify.max_user_watches
```

O valor 524288 corresponde a 512 mil watches e atende a maioria dos cenários de produção. O ajuste deve ocorrer antes de expandir o conjunto de diretórios monitorados, e não depois que os eventos param, pois a ausência de eventos é a única evidência visível do limite atingido. A regra nativa 560, descrita no capítulo 7, alerta quando a fila inotify do kernel transborda.

CAMINHOS COM /*: Um path terminado em `/*` em `<directories>` faz o `syscheckd` expandir o glob e registrar watches em arquivos individuais existentes, sem watch ativo para arquivos novos. Declare o diretório sem wildcard. Evidência no Achado AO-8 (capítulo 10).

COMO VERIFICAR SE O REALTIME ENTREGA EVENTOS: Confirme no manager, em `archives.json` ou no índice `wazuh-archives-*`, que os eventos chegam com `syscheck.mode` igual a `realtime`, mesmo quando o alerta não aparece em `wazuh-alerts-*`: uma regra filha em nível 0 suprime o alerta (seção 5.2). No agente, `strace -y -f -p` no `wazuh-syscheckd` deve mostrar `read()` no descritor `anon_inode:inotify` a cada alteração; inclua `pselect6` no filtro de syscalls, pois é a chamada de espera observada no `syscheckd`. Não consulte o `fim.db` com o agente ativo: o `sqlite3` retorna `database is locked`.

4.9 Checklist Consolidado de Troubleshooting

Em ambas as plataformas, a maioria dos relatos de "o FIM não gera os eventos esperados" se resolve em uma lista curta de causas. Verificar nesta ordem costuma ser o caminho mais rápido.

1. Confirmar que o caminho está declarado no bloco `<syscheck>` efetivo, localmente ou via `agent.conf` de grupo, com os atributos que o requisito exige (`check_all`, `whodata`, `report_changes`).
2. No Windows, confirmar que o caminho é local, nunca UNC ou drive mapeado. A restrição vale para todo modo desde a 4.13.0.
3. Confirmar que o serviço do agente foi reiniciado após a mudança. O syscheckd não aplica novos diretórios em reload a quente.
4. No Windows, validar a política de auditoria efetiva com `auditpol` por GUID, nunca pelo `secpol.msc`, e confirmar Êxito nas duas subcategorias.
5. No Windows domain joined, rodar `gpresult /h` e procurar GPO que defina Sistema de Arquivos ou Manipulação de Identificador.
6. No Linux, confirmar auditd instalado e ativo, regras presentes (`auditctl -l | grep wazuh_fim`) e `restart_audit` não desabilitado.
7. No Linux, com auditd ativo e whodata inativo, procurar "Address already in use" do `audisp-af_unix` no `journalctl` e remover o socket órfão.
8. No Linux, checar perda de eventos (`auditctl -s | grep lost`) e persistência das regras em `/etc/audit/rules.d/`.
9. No Linux, comparar `max_user_watches` com o número de watches em uso e conferir paths sem `/*`.
10. Revisar o `ossec.log` do agente em busca do health check do whodata e do fallback para realtime, buscando "Whodata" sem hífen.

As Figuras 16 a 19 reorganizam o checklist acima como uma árvore de decisão para a pergunta "por que o evento de FIM esperado não chegou?". A Figura 16 reúne as checagens comuns às duas plataformas e direciona para a figura do sintoma: endpoint Windows, endpoint Linux ou manager. Cada cartão traz a verificação, a resposta que dispara a ação e a referência da seção ou do achado deste handbook.

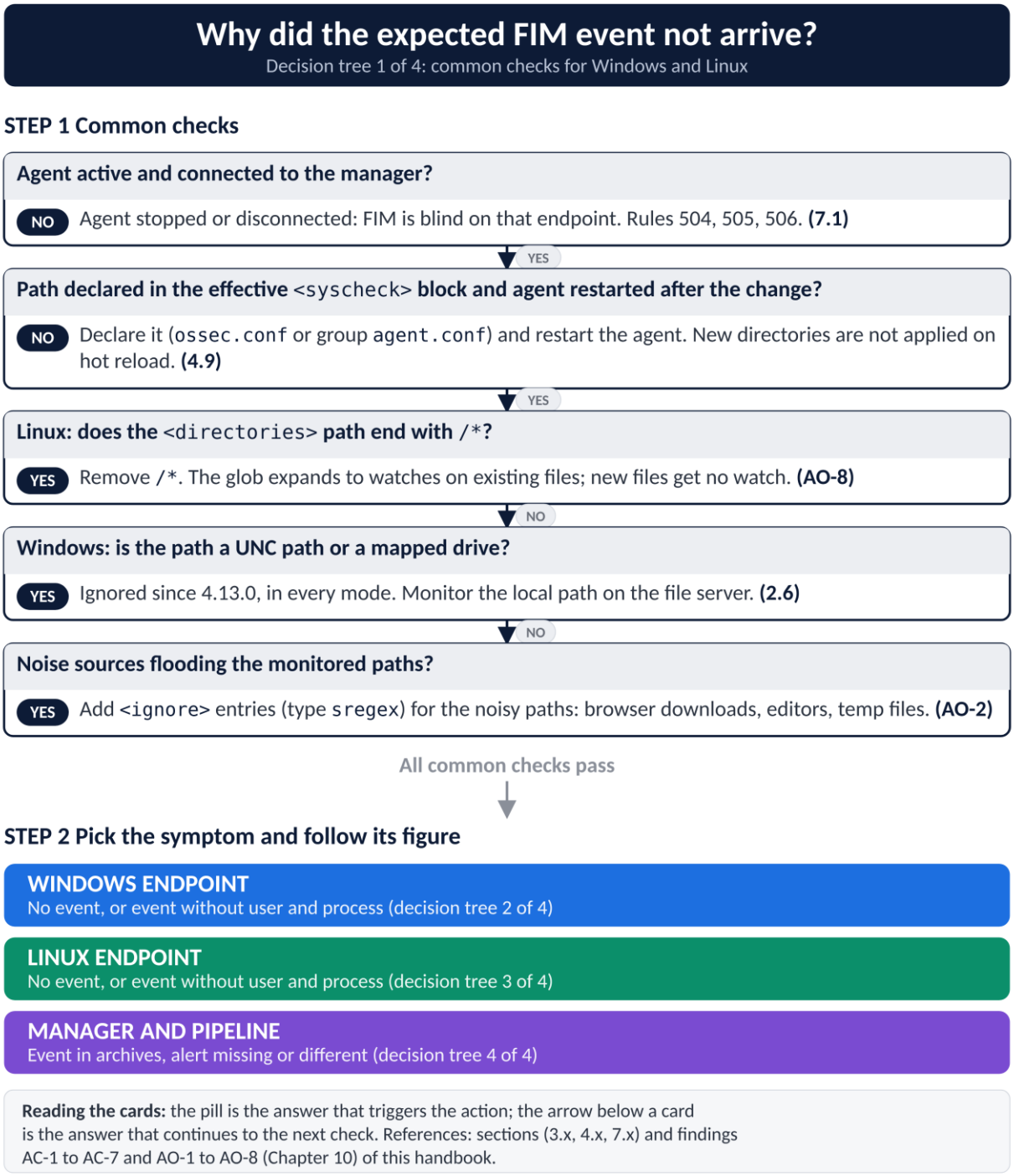


Figura 16 - Árvore de decisão (1 de 4): checagens comuns às duas plataformas e seletor de sintoma

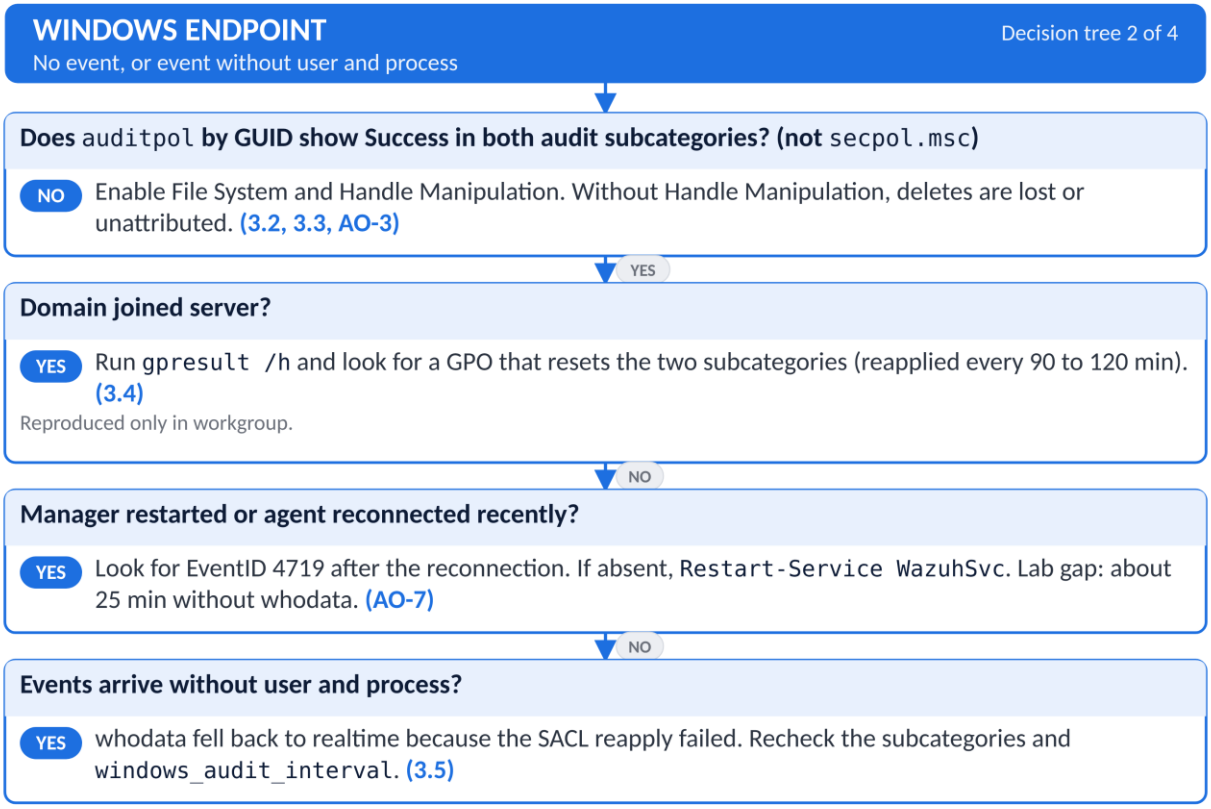


Figura 17 - Árvore de decisão (2 de 4): endpoint Windows (política de auditoria, GPO, SACL após restart do manager e fallback do whodata)

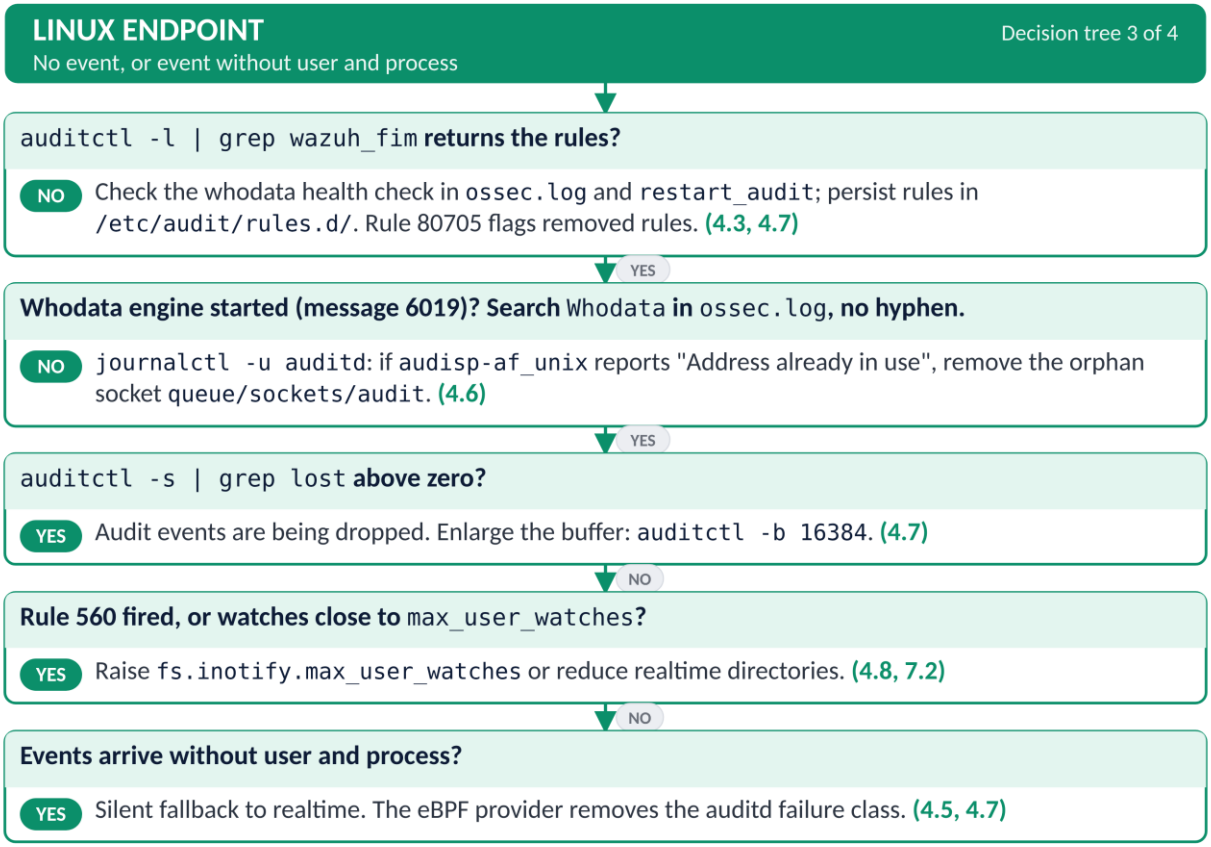


Figura 18 - Árvore de decisão (3 de 4): endpoint Linux (regras de auditoria, motor whodata, socket órfão, eventos perdidos e inotify)

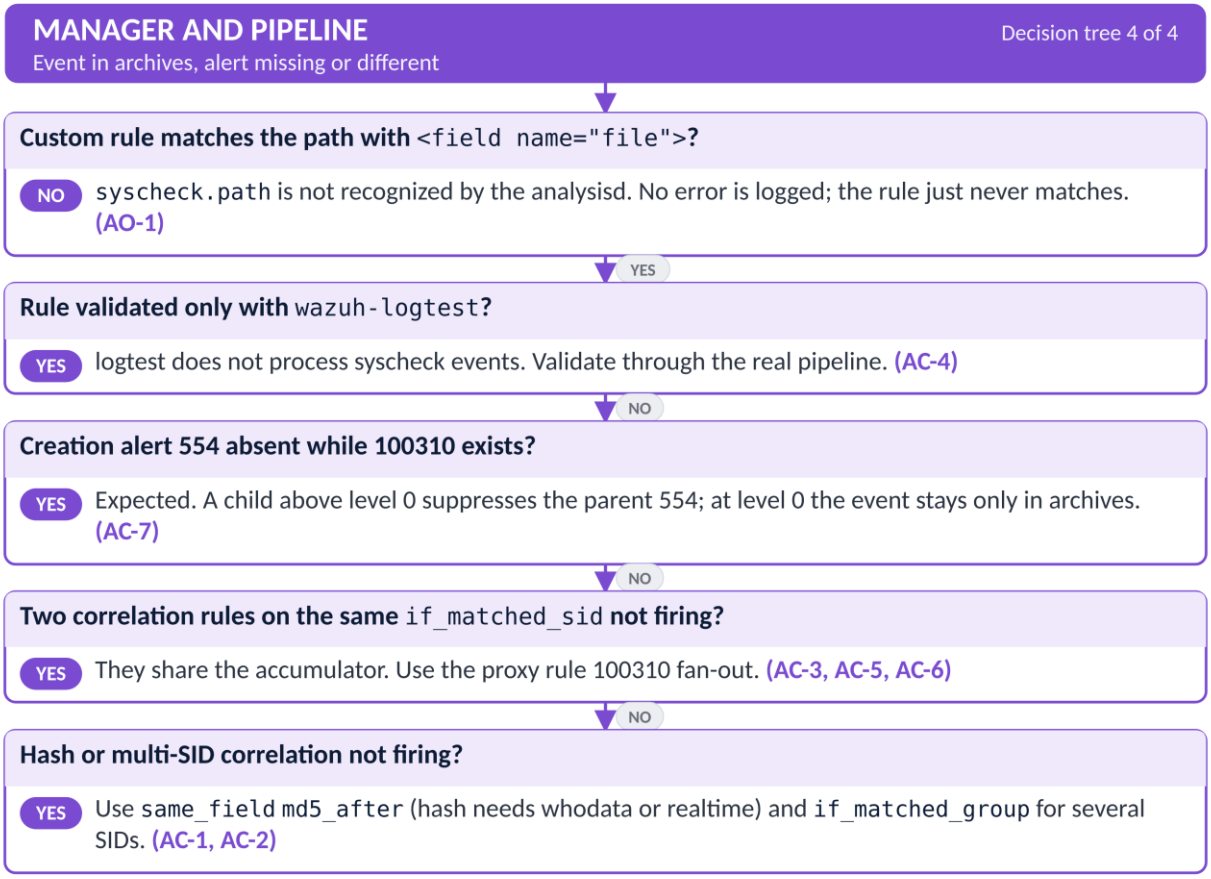


Figura 19 - Árvore de decisão (4 de 4): manager e pipeline (campo da regra, logtest, proxy 100310 e contadores de correlação)

5. Regras de Detecção de Anomalia

O FIM nativo detecta eventos individuais. Padrões comportamentais, como criação em massa, exclusão em massa ou replicação de um mesmo binário, exigem correlação. As seis regras deste capítulo, definidas em `fim_anomaly_rules.xml`, estendem as regras base 550, 553 e 554 por correlação temporal e cobrem os cenários de atividade suspeita detectáveis exclusivamente via FIM, sem dependência de Sysmon ou de qualquer telemetria adicional.

AVISO DE THRESHOLD: Os valores `frequency="3"` e `timeframe="600"` (10 minutos) foram calibrados para teste em laboratório. Em produção, especialmente em servidores de arquivo, hosts de compilação, sistemas de backup e repositórios de documentos, ajuste-os ao perfil legítimo de cada grupo de agentes antes do deploy. O `ignore="30"` também é valor de laboratório; em produção, `ignore="0"` garante cobertura sem supressão de alertas.

5.1 Inventário das Regras

ID	Nível	Tipo	Correlação	Gatilho	MITRE
100310	5	Filha/proxy	`if_sid` 554	Each `syscheck_new_entry` (554) is re-emitted to groups `fim_creation` and `fim_new_entry`	-
100300	10	Correlação	`if_matched_group fim_creation`	3 or more files created within 10 min on the same agent	T1565, T1565.001, T1074
100301	12	Correlação	`if_matched_group fim_new_entry`	Same MD5 (`md5_after`) in 3 or more directories within 10 min	T1105, T1570
100302	10	Correlação	`if_matched_sid` 550	3 or more files modified within 10 min	T1565, T1565.001
100303	10	Correlação	`if_matched_sid` 553	3 or more files deleted within 10 min	T1485, T1070, T1561
100304	14	Escalação	`if_matched_group syscheck_file`	5 or more mixed events (create, modify, delete) within 10 min	T1485, T1486, T1565

Tabela 11 - Regras customizadas de FIM (arquivo /var/ossec/etc/rules/fim_anomaly_rules.xml, permissões 660 wazuh:wazuh)

5.2 Arquitetura de Contadores e a Regra Proxy 100310

A regra 100310 é o elemento arquitetural central. O analysisd mantém contadores de correlação por regra base; quando duas regras de correlação usam `if_matched_sid` sobre o mesmo SID, elas compartilham o acumulador e interferem entre si (Achados AC-3, AC-5 e AC-6). A solução é um proxy de aplicação: a regra 100310, filha de 554, reencaminha cada evento 554 para dois grupos, criando dois acumuladores independentes. É o padrão de fan-out de arquiteturas orientadas a eventos.



Figura 20 - Fluxo de correlação: a regra 100310 isola os contadores de 100300 e 100301; 100302 e 100303 correlacionam direto sobre 550 e 553

> **if_sid e if_matched_sid são mecanismos distintos:** `<if_sid>` define uma regra filha: ela só dispara se o SID especificado acabou de disparar no mesmo evento. `<if_matched_sid>` define correlação temporal: o analysisd mantém um contador independente e dispara quando `frequency` eventos do SID ocorrem dentro do `timeframe`.
>
> A 100310 usa `<if_sid>554` para interceptar cada evento 554 e emiti-lo aos grupos `fim_creation` e `fim_new_entry`. As regras 100300 e 100301 usam `<if_matched_group>` sobre esses grupos para contagem independente. Sem a 100310, ambas precisariam de `<if_matched_sid>554` e compartilhariam o contador.

> **EFEITO COLATERAL DA FILHA (ACHADO AC-7):** Com a 100310 em nível maior que zero, o analysisd suprime o alerta da regra pai 554: em `wazuh-alerts-*` aparece apenas `rule.id=100310`, nunca 554. Em nível 0, o alerta é suprimido por completo e o evento aparece somente em `wazuh-archives-*`, sem campo `rule`. Por isso a 100310 opera em nível 5 e o ID 100310 consta em `FIM_FILE_RULES` do painel de investigação (capítulo 6).

5.3 Definição das Regras

Conteúdo vigente do arquivo `fim_anomaly_rules.xml`, com a 100310 em nível 5 conforme validado em 10/10/2026. As descrições permanecem em inglês, como no arquivo implantado.

```
<group name="syscheck,fim_anomaly,">


  <rule id="100310" level="5">
    <if_sid>554</if_sid>
    <description>FIM: New file detected (syscheck_new_entry) [base: rule 554]. Event forwarded to
fim_creation and fim_new_entry groups for independent counter isolation of rules 100300 and
100301.</description>
    <group>fim_creation,fim_new_entry,</group>
  </rule>


  <rule id="100300" level="10" frequency="3" timeframe="600" ignore="30">
    <if_matched_group>fim_creation</if_matched_group>
    <description>FIM: 3 or more files created within 10 minutes. Possible malware staging or automated
write activity. Lab threshold - tune for production.</description>
    <group>suspicious_creation,</group>
    <mitre><id>T1565</id><id>T1565.001</id><id>T1074</id></mitre>
  </rule>


  <rule id="100301" level="12" frequency="3" timeframe="600" ignore="30">
    <if_matched_group>fim_new_entry</if_matched_group>
    <same_field>md5_after</same_field>
    <different_field>file</different_field>
    <description>FIM: Same file hash created in 3 or more different directories within 10 minutes.
Possible malware propagation or dropper replication. Lab threshold - tune for production.</description>
    <group>suspicious_creation,</group>
    <mitre><id>T1105</id><id>T1570</id></mitre>
  </rule>


  <rule id="100302" level="10" frequency="3" timeframe="600" ignore="30">
    <if_matched_sid>550</if_matched_sid>
    <description>FIM: 3 or more files modified within 10 minutes. Possible payload execution, in-place
encryption, or automated tampering. Lab threshold - tune for production.</description>
    <group>suspicious_modification,</group>
    <mitre><id>T1565</id><id>T1565.001</id></mitre>
  </rule>


  <rule id="100303" level="10" frequency="3" timeframe="600" ignore="30">
    <if_matched_sid>553</if_matched_sid>
    <description>FIM: 3 or more files deleted within 10 minutes. Possible evidence destruction, anti-
forensics, or post-execution malware cleanup. Lab threshold - tune for production.</description>
```

```
    <group>suspicious_deletion,</group>
    <mitre><id>T1485</id><id>T1070</id><id>T1561</id></mitre>
  </rule>


  <rule id="100304" level="14" frequency="5" timeframe="600" ignore="30">
    <if_matched_group>syscheck_file</if_matched_group>
    <description>FIM: High volume mixed file activity - 5 or more create, modify, and delete events
within 10 minutes. Combined pattern suggesting active malware execution. Lab threshold - tune for
production.</description>
    <group>suspicious_combined,fim_escalation,</group>
    <mitre><id>T1485</id><id>T1486</id><id>T1565</id></mitre>
  </rule>


</group>
```

Deploy

```
chown wazuh:wazuh fim_anomaly_rules.xml
chmod 660 fim_anomaly_rules.xml
cp fim_anomaly_rules.xml /var/ossec/etc/rules/
/var/ossec/bin/wazuh-analysisd -t 2>&1 | grep -i error
systemctl restart wazuh-manager
```

Notas de uso

- **Escopo por diretório:** em deployments com vários paths, restrinja a 100300 por diretório com um filtro `<field name="file" type="pcre2">(?i)/caminho/alvo</field>`. Em regras FIM customizadas, o campo de path correto é `file`, e não `syscheck.path` (Achado AO-1).
- **Hash em 100301:** o campo de correlação é `md5_after`, não `md5` (Achado AC-1).
- **Múltiplos SIDs:** a 100304 usa `if_matched_group` sobre `syscheck_file`, grupo das regras 550, 553 e 554 em `0015-ossec_rules.xml`, porque `if_matched_sid` com SIDs separados por vírgula não é confiável (Achado AC-2).
- **Cooldown ignore:** no laboratório, o `ignore` se mostrou global por regra, e não por agente: o disparo no agente A suprimiu a mesma regra no agente B dentro do período. Com agentes de perfis distintos, considere `ignore="0"` e controle o volume por grupos de agentes e thresholds por grupo.
- **Validação:** o `wazuh-logtest` não processa eventos syscheck (Achado AC-4); valide sempre pelo pipeline real.

5.4 Cenário 1: Criação de Arquivos (554, 100310, 100300, 100301)

A regra base 554 (`syscheck_new_entry`, nível 5) dispara para cada arquivo novo. A 100310 a substitui no pipeline de alertas (Achado AC-7). A 100300 detecta criação em massa: indica staging de malware, execução automatizada ou escrita massiva de artefatos por processo malicioso. A 100301, a de maior nível do grupo, detecta o mesmo conteúdo (MD5 idêntico) em três ou mais diretórios distintos: replicação de dropper ou staging lateral de ferramentas.

Os scripts de simulação criaram quatro arquivos com intervalo de 3 segundos entre cada operação, mantendo os limiares de correlação dentro da janela de 10 minutos.

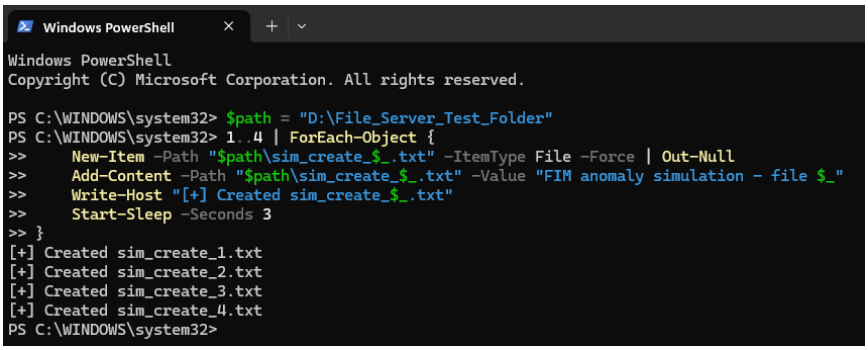


Figura 21 - Script PowerShell de simulação de criação de arquivos no LABDESK (D:\File_Server_Test_Folder)

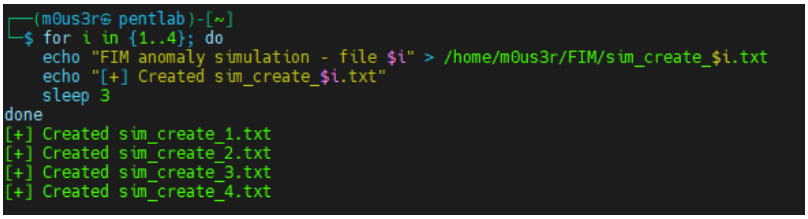


Figura 22 - Script Bash de simulação de criação de arquivos no pentlab (/home/m0us3r/FIM)



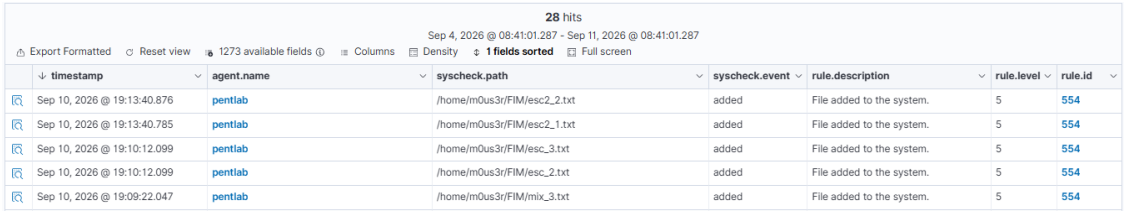
30 hits

Sep 4, 2026 @ 08:31:02.016 - Sep 11, 2026 @ 08:31:02.016

Export Formatted | Reset view | 1273 available fields | Columns | Density | **1 fields sorted** | Full screen

timestamp	agent.name	syscheck.path	syscheck.event	rule.description	rule.level	rule.id
Sep 10, 2026 @ 22:25:09.367	LABDESK	d:\file_server_test_folder\t99_4.txt	added	File added to the system.	5	554
Sep 10, 2026 @ 22:25:05.338	LABDESK	d:\file_server_test_folder\t99_2.txt	added	File added to the system.	5	554
Sep 10, 2026 @ 22:25:03.333	LABDESK	d:\file_server_test_folder\t99_1.txt	added	File added to the system.	5	554
Sep 10, 2026 @ 22:18:11.611	LABDESK	d:\file_server_test_folder\tst4_6.txt	added	File added to the system.	5	554
Sep 10, 2026 @ 22:18:09.609	LABDESK	d:\file_server_test_folder\tst4_5.txt	added	File added to the system.	5	554

Figura 23 - Regra 554: 30 eventos de criação (syscheck.event added, nível 5) no LABDESK



28 hits

Sep 4, 2026 @ 08:41:01.287 - Sep 11, 2026 @ 08:41:01.287

Export Formatted | Reset view | 1273 available fields | Columns | Density | **1 fields sorted** | Full screen

timestamp	agent.name	syscheck.path	syscheck.event	rule.description	rule.level	rule.id
Sep 10, 2026 @ 19:13:40.876	pentlab	/home/m0us3r/FIM/esc2_2.txt	added	File added to the system.	5	554
Sep 10, 2026 @ 19:13:40.785	pentlab	/home/m0us3r/FIM/esc2_1.txt	added	File added to the system.	5	554
Sep 10, 2026 @ 19:10:12.099	pentlab	/home/m0us3r/FIM/esc_3.txt	added	File added to the system.	5	554
Sep 10, 2026 @ 19:10:12.099	pentlab	/home/m0us3r/FIM/esc_2.txt	added	File added to the system.	5	554
Sep 10, 2026 @ 19:09:22.047	pentlab	/home/m0us3r/FIM/mix_3.txt	added	File added to the system.	5	554

Figura 24 - Regra 554: 28 eventos de criação no pentlab (Linux)

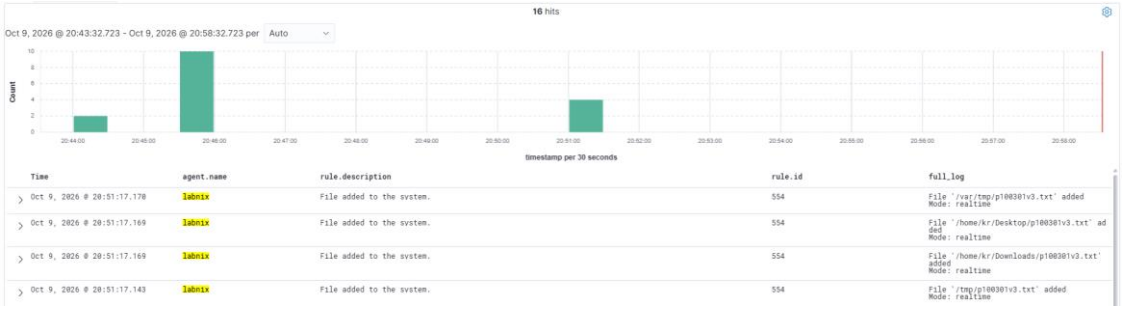


Figura 25 - Regra 554: 16 eventos de criação no labnix (Linux, Ubuntu 24.04.5), 09/10/2026; o arquivo p100301v3.txt foi criado às 20:51:17 em /var/tmp, /home/kr/Desktop, /home/kr/Downloads e /tmp, com Mode: realtime



Figura 26 - Regra 100310 (proxy, filha de 554) com a descrição "[base: rule 554]" no labnix, 10/10/2026 13:26:17

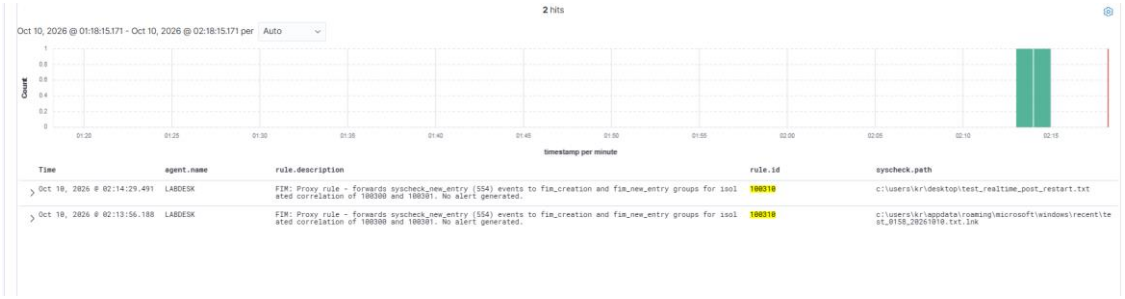


Figura 27 - Regra 100310 no LABDESK, 10/10/2026: criação de test_realtime_post_restart.txt no Desktop (02:14:29) e de um atalho .lnk em Recent (02:13:56); a descrição exibida é a anterior à atualização com [base: rule 554]



Figura 28 - Regra 100310 no labnix, 10/10/2026: os hits de 02:30:14 e 02:31:41 exibem a descrição anterior ("Proxy rule ... No alert generated") e o de 02:35:52 já exibe a descrição atualizada ("New file detected (syscheck_new_entry)")

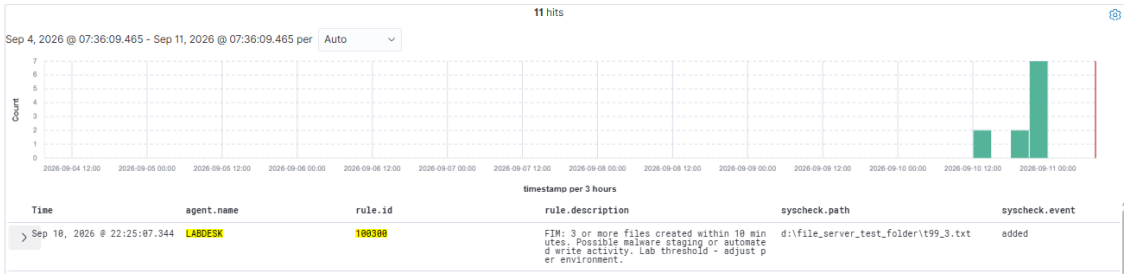


Figura 29 - Regra 100300 disparada no LABDESK (Windows), syscheck.event added

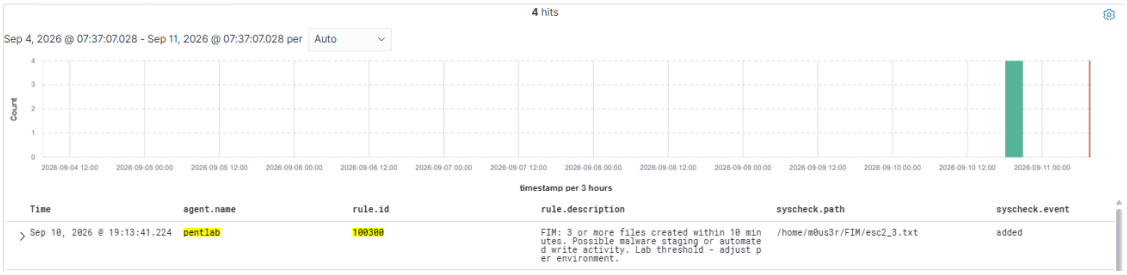


Figura 30 - Regra 100300 disparada no pentlab (Linux), syscheck.event added



Figura 31 - Regra 100300 disparada no labnix (Linux, Ubuntu 24.04.5): syscheck.event added em /var/tmp/payload_test.txt, 09/10/2026 20:18:07; 22 ocorrências em 24 horas

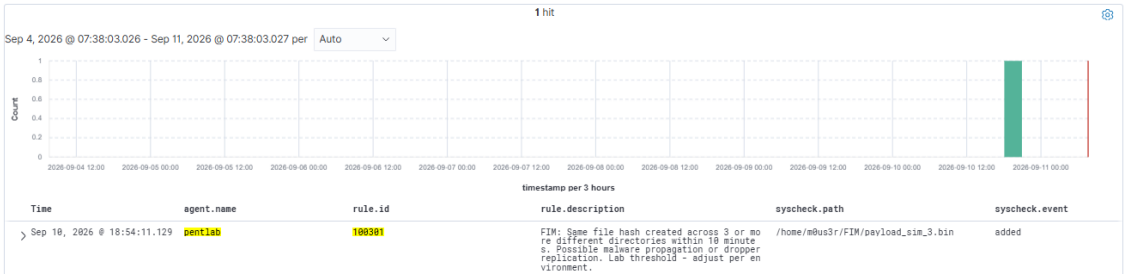


Figura 32 - Regra 100301 disparada no pentlab: mesmo hash MD5 (md5_after) em diretórios distintos, arquivo payload_sim_3.bin

5.5 Cenário 2: Modificação de Arquivos (550, 100302)

A regra base 550 (`syscheck_integrity_changed`, nível 7) registra o caminho e os hashes antes e depois da modificação. A 100302 dispara com 3 ou mais modificações em 10 minutos e correlaciona direto sobre `if_matched_sid` 550, sem regra proxy, porque nenhuma regra filha de 550 interfere no contador. Indica execução de payload, criptografia in-place (padrão inicial de ransomware) ou adulteração automatizada de configuração. No Linux, com `frequency` de 60 s, o scan agendado pode gerar eventos 550 por mudança de mtime em arquivos recém-criados: comportamento esperado em laboratório, e não em produção com a frequência padrão de 43200 s.

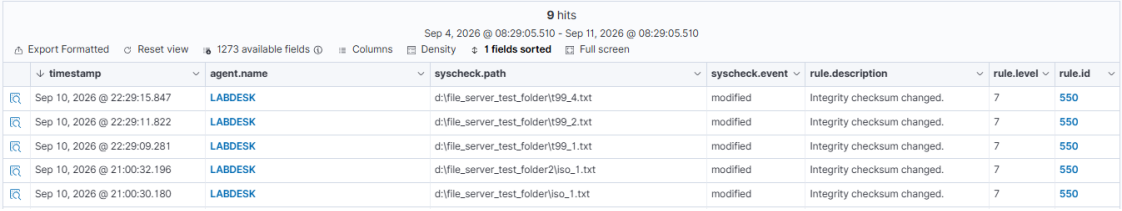


Figura 33 - Regra 550: 9 eventos de modificação (nível 7) no LABDESK

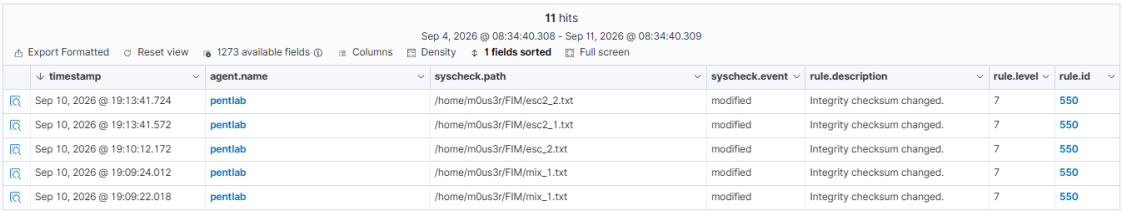


Figura 34 - Regra 550: 11 eventos de modificação no pentlab

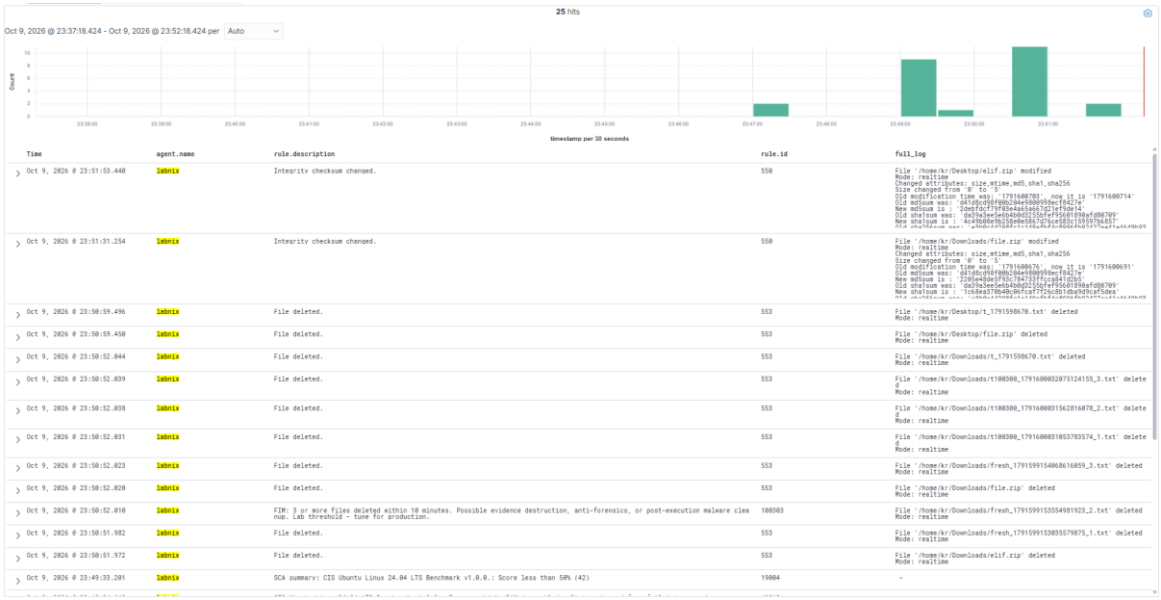


Figura 35 - Regra 550 no labnix, 09/10/2026 23:51: modificações de /home/kr/Desktop/elif.zip e /home/kr/Downloads/file.zip (Mode: realtime; alteração de size, mtime, md5, sha1 e sha256), seguidas das exclusões em sequência (553); 25 hits

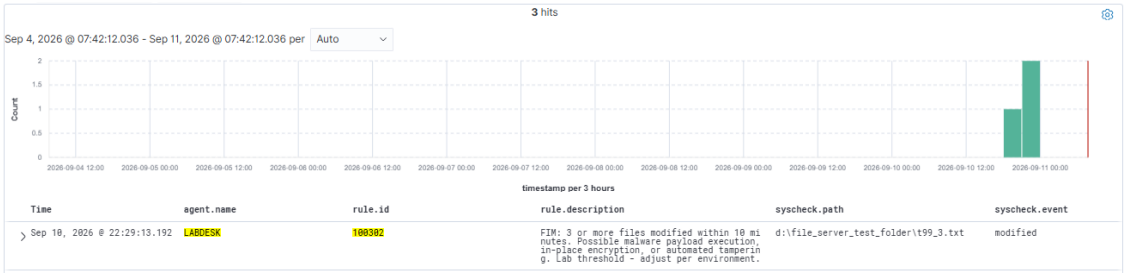


Figura 36 - Regra 100302 disparada no LABDESK, syscheck.event modified

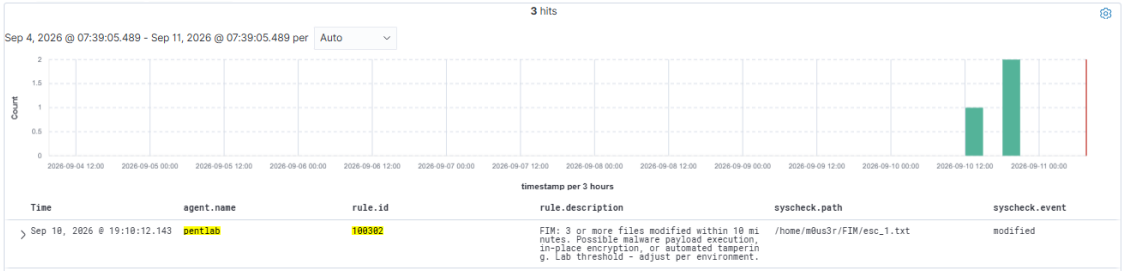


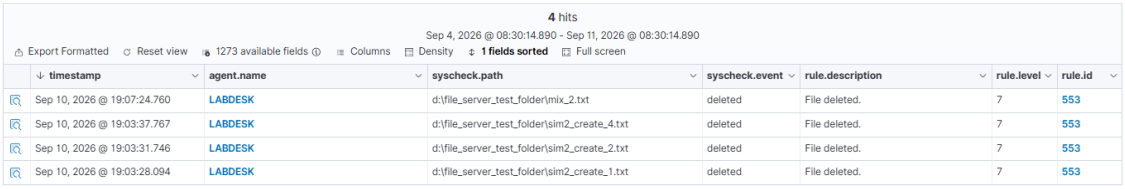
Figura 37 - Regra 100302 disparada no pentlab, syscheck.event modified



Figura 38 - Regra 100302 disparada no labnix (Linux, Ubuntu 24.04.5): /home/kr/Downloads/beacon_test.sh modificado (Mode: realtime; size, md5, sha1 e sha256 alterados), 08/10/2026 12:30:12

5.6 Cenário 3: Exclusão de Arquivos (553, 100303)

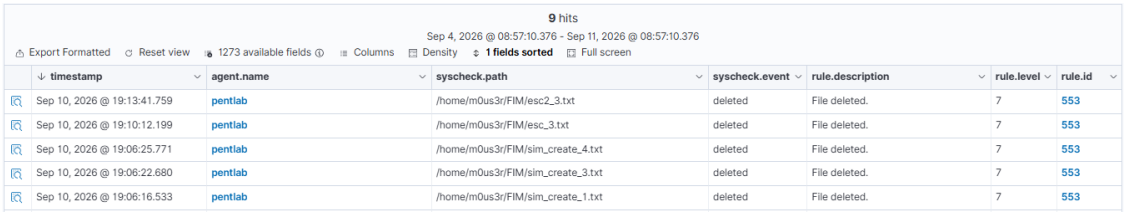
A regra base 553 (`syscheck_deleted`, nível 7) registra a exclusão. A 100303 dispara com 3 ou mais exclusões em 10 minutos e não sofre interferência de contador, porque não há regra filha associada ao SID 553. Indica destruição de evidências, anti-forense, remoção de logs ou limpeza pós-execução de malware.



4 hits
Sep 4, 2026 @ 08:30:14.890 - Sep 11, 2026 @ 08:30:14.890
Export Formatted | Reset view | 1273 available fields | Columns | Density | 1 fields sorted | Full screen

timestamp	agent.name	syscheck.path	syscheck.event	rule.description	rule.level	rule.id
Sep 10, 2026 @ 19:07:24.760	LABDESK	d:\file_server_test_folder\mix_2.txt	deleted	File deleted.	7	553
Sep 10, 2026 @ 19:03:37.767	LABDESK	d:\file_server_test_folder\sim2_create_4.txt	deleted	File deleted.	7	553
Sep 10, 2026 @ 19:03:31.746	LABDESK	d:\file_server_test_folder\sim2_create_2.txt	deleted	File deleted.	7	553
Sep 10, 2026 @ 19:03:28.094	LABDESK	d:\file_server_test_folder\sim2_create_1.txt	deleted	File deleted.	7	553

Figura 39 - Regra 553: 4 eventos de exclusão (nível 7) no LABDESK



9 hits
Sep 4, 2026 @ 08:57:10.376 - Sep 11, 2026 @ 08:57:10.376
Export Formatted | Reset view | 1273 available fields | Columns | Density | 1 fields sorted | Full screen

timestamp	agent.name	syscheck.path	syscheck.event	rule.description	rule.level	rule.id
Sep 10, 2026 @ 19:13:41.759	pentlab	/home/m0us3r/FIM/esc2_3.txt	deleted	File deleted.	7	553
Sep 10, 2026 @ 19:10:12.199	pentlab	/home/m0us3r/FIM/esc_3.txt	deleted	File deleted.	7	553
Sep 10, 2026 @ 19:06:25.771	pentlab	/home/m0us3r/FIM/sim_create_4.txt	deleted	File deleted.	7	553
Sep 10, 2026 @ 19:06:22.680	pentlab	/home/m0us3r/FIM/sim_create_3.txt	deleted	File deleted.	7	553
Sep 10, 2026 @ 19:06:16.533	pentlab	/home/m0us3r/FIM/sim_create_1.txt	deleted	File deleted.	7	553

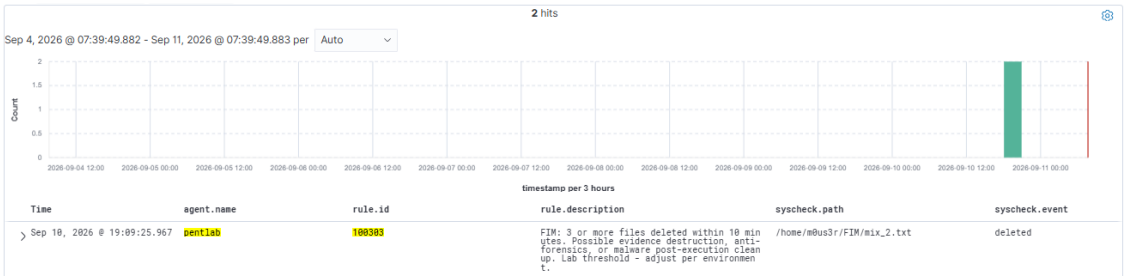
Figura 40 - Regra 553: 9 eventos de exclusão no pentlab



34 hits
Oct 9, 2026 @ 22:16:10.758 - Oct 9, 2026 @ 23:16:10.758 per Auto
Count
timestamp per minute

Oct 9, 2026 @ 23:15:45.059	labnix	File deleted.	553	File '/home/kr/Desktop/file.zip' deleted Mode: realtime
Oct 9, 2026 @ 23:15:21.014	labnix	File deleted.	553	File '/home/kr/Downloads/t_1791597370.txt' deleted Mode: realtime
Oct 9, 2026 @ 23:15:21.005	labnix	File deleted.	553	File '/home/kr/Downloads/TESTING.txt' deleted Mode: realtime
Oct 9, 2026 @ 23:15:21.002	labnix	File deleted.	553	File '/home/kr/Downloads/t_1791591307.txt' deleted Mode: realtime
Oct 9, 2026 @ 23:15:21.000	labnix	File deleted.	553	File '/home/kr/Downloads/payload_test.txt' deleted Mode: realtime
Oct 9, 2026 @ 23:15:20.996	labnix	File deleted.	553	File '/home/kr/Downloads/p100301v4.txt' deleted Mode: realtime
Oct 9, 2026 @ 23:15:20.989	labnix	File deleted.	553	File '/home/kr/Downloads/p5.txt' deleted Mode: realtime
Oct 9, 2026 @ 23:15:20.976	labnix	File deleted.	553	File '/home/kr/Downloads/p100301.txt' deleted Mode: realtime
Oct 9, 2026 @ 23:15:20.976	labnix	File deleted.	553	File '/home/kr/Downloads/p100301v3.txt' deleted Mode: realtime
Oct 9, 2026 @ 23:15:16.127	labnix	FIM: 3 or more files deleted within 10 minutes. Possible evidence destruction, anti-forensics, or post-executio n malware cleanup. Lab threshold - tune for production.	100303	File '/home/kr/Downloads/file.zip' deleted Mode: realtime
Oct 9, 2026 @ 23:08:33.257	labnix	File deleted.	553	File '/tmp/tmp.W1S71uGJAV' deleted Mode: realtime
Oct 9, 2026 @ 23:08:33.256	labnix	FIM: 3 or more files deleted within 10 minutes. Possible evidence destruction, anti-forensics, or post-executio n malware cleanup. Lab threshold - tune for production.	100303	File '/tmp/tmp.r5QKE8X2Tz' deleted Mode: realtime
Oct 9, 2026 @ 23:08:33.256	labnix	File deleted.	553	File '/tmp/tmp.V4ey0ZXe1t' deleted Mode: realtime
Oct 9, 2026 @ 23:08:33.230	labnix	Integrity checksum changed.	550	File '/tmp/tmp.V4ey0ZXe1t' modified Mode: realtime Changed attributes: size,mtime,md5,sha1,sha256 Size changed from '0' to '100' Old modification time was: '1791598112', now it is '179159 8113' Old md5sum was: 'd41d8cd98f00b204e9800998ecf8427e'

Figura 41 - Regra 553 no labnix, 09/10/2026 de 23:08 a 23:15: exclusões em /home/kr/Downloads, /home/kr/Desktop e /tmp (Mode: realtime), com a 100303 disparando às 23:08:33 e às 23:15:16; 34 hits



2 hits
Sep 4, 2026 @ 07:39:49.882 - Sep 11, 2026 @ 07:39:49.883 per Auto
Count
timestamp per 3 hours

Time	agent.name	rule.id	rule.description	syscheck.path	syscheck.event
Sep 10, 2026 @ 19:09:25.967	pentlab	100303	FIM: 3 or more files deleted within 10 minutes. Possible evidence destruction, anti-forensics, or malware post-execution clean up. Lab threshold - adjust per environmen t.	/home/m0us3r/FIM/mix_2.txt	deleted

Figura 42 - Regra 100303 disparada no pentlab, syscheck.event deleted

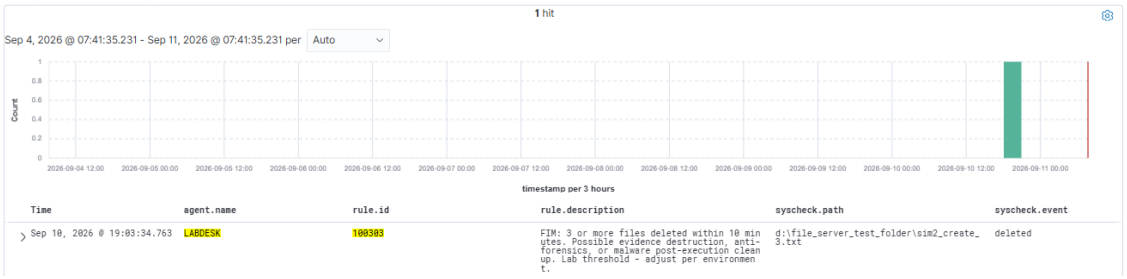


Figura 43 - Regra 100303 disparada no LABDESK, syscheck.event deleted

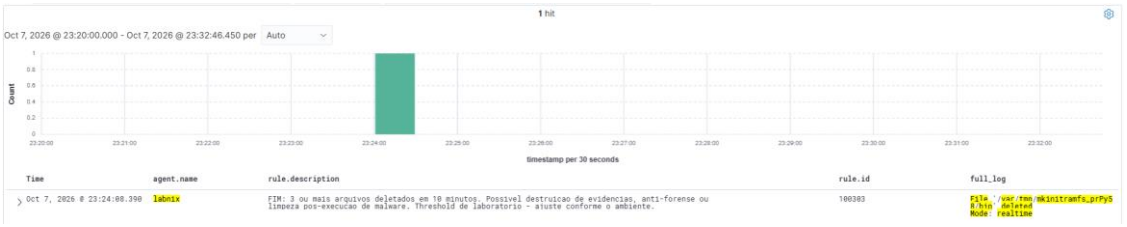


Figura 44 - Regra 100303 disparada no labnix (Linux, Ubuntu 24.04.5): exclusão de /var/tmp/mkinitramfs_prPy58/bin (Mode: realtime), 07/10/2026 23:24:08

5.7 Escalação: Atividade Mista de Alto Volume (100304)

A 100304 é a regra de escalação, nível 14, e exige atenção imediata. Dispara com 5 ou mais eventos de qualquer combinação de criação, modificação e exclusão em 10 minutos no mesmo agente. O padrão criar-modificar-excluir ou escrever-depois-excluir é um indicador comportamental forte de execução ativa de malware.

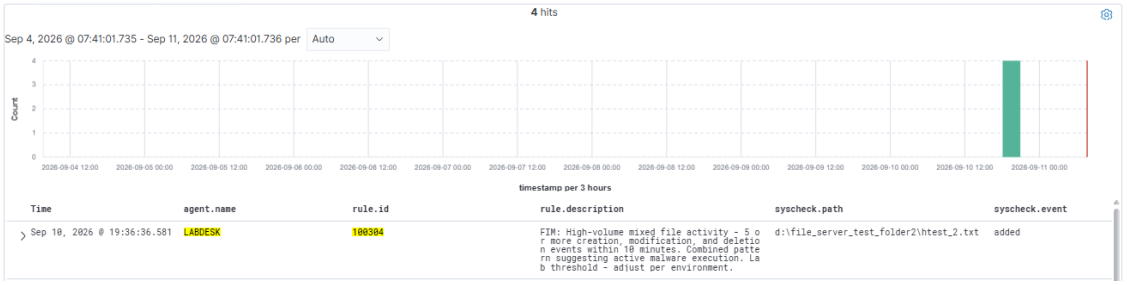


Figura 45 - Regra 100304 (nível 14) disparada no LABDESK: atividade mista dentro de 10 minutos

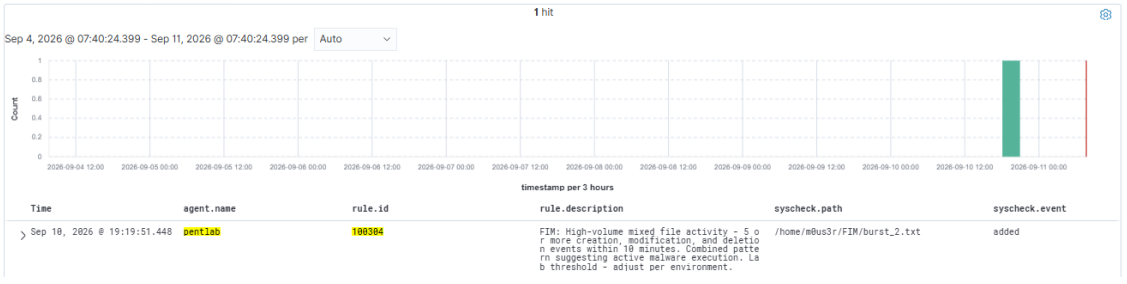


Figura 46 - Regra 100304 (nível 14) disparada no pentlab

5.8 Matriz de Validação

Regra	Cenário	LABDESK (Windows)	pentlab (Linux)	Observação
100310	Isolation proxy (level 5)	**CONFIRMADO**	-	Validada em LABDESK e labnix (Linux) em 10/10/2026

Regra	Cenário	LABDESK (Windows)	pentlab (Linux)	Observação
100300	3+ creations within 10 min	**CONFIRMADO**	**CONFIRMADO**	Usa `fim_creation` via 100310
100301	Same hash, different directories	**PARCIAL**	**CONFIRMADO**	Windows: `same_field` sobre campos syscheck não acessível ao motor de correlação nos testes de 16/09/2026
100302	3+ modifications within 10 min	**CONFIRMADO**	**CONFIRMADO**	`if_matched_sid` 550
100303	3+ deletions within 10 min	**CONFIRMADO**	**CONFIRMADO**	`if_matched_sid` 553
100304	5+ mixed events (level 14)	**CONFIRMADO**	**CONFIRMADO**	`if_matched_group` `syscheck_file`

Tabela 12 - Matriz de validação em laboratório. Endpoints Windows mantêm cobertura de criação via 100300 e de escalação via 100304

> **Escopo da validação:** As evidências deste capítulo foram obtidas com os scripts de simulação e os alertas reais indexados em `wazuh-alerts-*`. Testes formais de threshold controlado das regras 100300 a 100304, com cenários documentados, estão fora do escopo deste handbook.

6. FIM Alert Investigation

O FIM Alert Investigation é um componente comunitário construído para esta pesquisa, entregue como o plugin WPTV para o Wazuh Dashboard. Não faz parte da plataforma Wazuh e não tem suporte da Wazuh.

O **FIM Alert Investigation** é uma extensão do WAZUH-Process-Tree-Viewer (WPTV), exposta na aba **FORENSICS** do Wazuh Dashboard. Seu objetivo é transformar um alerta FIM bruto em contexto acionável antes de o analista abrir o terminal: em vez de navegar manualmente entre índices no Discover, o analista seleciona o alerta, aciona a análise e recebe em segundos um veredito estruturado com `risk_score`, `risk_level`, técnicas MITRE com percentual de confiança, IOCs e ações recomendadas.

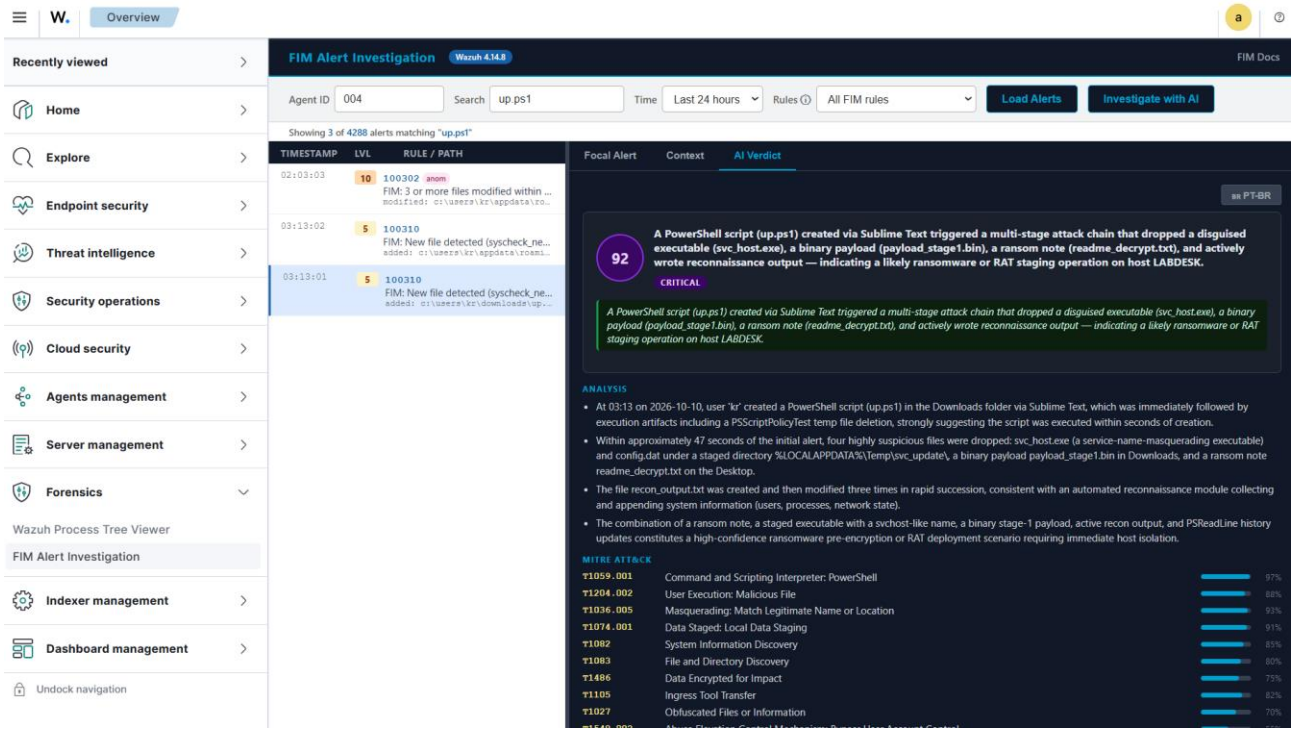


Figura 47 - FIM Alert Investigation interface (Aba Forensics)

> **Princípio de design:** A IA é assistida, não autônoma. O LLM contextualiza e recomenda; o analista decide e executa. Nenhuma ação automática sobre o endpoint é disparada pelo veredito.

6.1 Arquitetura

O componente opera inteiramente no servidor (WZ-SRV); nenhum componente adicional é instalado nos endpoints. O nginx roteia `/fim/` e `/api/fim/*` para o mesmo processo Flask (`server.py`, porta 5000) que já serve o WPTV em `/wptv/` e `/api/wptv/*`. Não há processo separado, porta dedicada nem serviço systemd próprio para o FIM Alert Investigation.

Rota nginx	Destino	Função
`/wptv/`	Flask :5000	WPTV: árvore de processos
`/fim/`	Flask :5000	FIM Alert Investigation: painel
`/api/fim/*`	Flask :5000	Endpoints de API do FIM

Tabela 13 - Roteamento nginx do WPTV e do FIM Alert Investigation

Arquivo	Local	Função
`logic_fim.py`	`process_tree_api/`	Lógica de investigação FIM (extensão do WPTV)
`server.py`	`process_tree_api/`	Servidor Flask com rotas `/api/fim/*`
`fim.html`	`process_tree_api/public/`	Painel FIM Alert Investigation

Tabela 14 - Arquivos do componente, sob /usr/share/wazuh-dashboard/plugins/process_tree_api/

6.2 Fluxo de Investigação

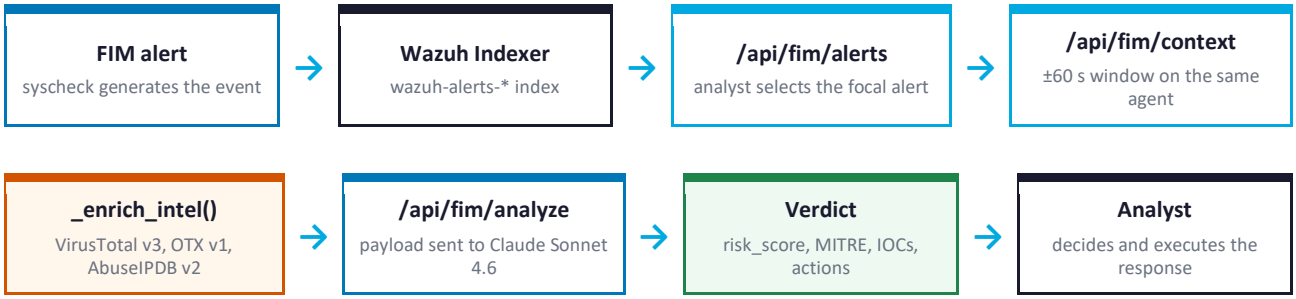


Figura 48 - Fluxo do FIM Alert Investigation: do alerta FIM ao veredito estruturado entregue ao analista

6.3 Contexto de ±60 Segundos

O contexto montado por `logic_fim.py` agrega, para o agente do alerta focal, quatro conjuntos de eventos indexados no Wazuh Indexer.

Conjunto	Conteúdo
`FIM_FILE_RULES`	`{550, 553, 554, 517, 100310}`: eventos de arquivo e a mensagem nativa Syscheck Audit (517, que não é regra de evento de arquivo); o 100310 é o evento visível de criação (Achado AC-7)
`FIM_REGISTRY_RULES`	`[594, 597, 598, 750, 751, 752]`: eventos de Registro do Windows
`FIM_ANOMALY_RULES`	`[100300, 100301, 100302, 100303, 100304]`: correlações de anomalia
`PROCESS_EVENTS`	EventID 4688 no Windows; eventos auditd no Linux

Tabela 15 - Conjuntos de regras e eventos consultados na janela de contexto

6.4 Enriquecimento de Threat Intelligence

O método `_enrich_intel()` consulta três fontes externas para cada alerta antes de enviar o payload ao LLM.

Fonte	Consulta	Retorno
VirusTotal (API v3)	Hash SHA256 em `/files/{hash}`	Detecções (`vt_malicious` e `vt_total`), família e permalink
AlienVault OTX (API v1)	Hash SHA256 em `/indicators/file/{hash}/general`	Pulses associados; um pulse indica atribuição a atores ou campanhas conhecidas
AbuseIPDB (API v2)	IPs em `data.audit.addr` nos eventos de processo capturados pelo auditd	Abuse score (0 a 100), total de reports, país e ISP

Tabela 16 - Fontes de enriquecimento do FIM Alert Investigation

LIMITE DO ENRIQUECIMENTO DE IP: O AbuseIPDB só é consultado quando há um IP em `data.audit.addr` nos eventos de processo. IPs presentes no conteúdo de arquivos monitorados não são extraídos pelo pipeline. Em um cenário de beacon C2, o `connect()` do processo precisa ser capturado pelo auditd (Achado AO-5).

Para capturar o `connect()` em labnix, regras de auditd foram adicionadas e carregadas com `augenrules --load`, e confirmadas ativas com `auditctl -l`:

```
# /etc/audit/rules.d/net_connect.rules
-a always,exit -F arch=b64 -S connect -F uid>=1000 -k net_connect_user
-a always,exit -F arch=b32 -S connect -F uid>=1000 -k net_connect_user
-a always,exit -F arch=b64 -S connect -F uid=0 -k net_connect_root
-a always,exit -F arch=b32 -S connect -F uid=0 -k net_connect_root
```

6.5 Plugin do Wazuh Dashboard, Logs e Serviço

- **Plugin wptv versão 2.1.0:** o FIM Alert Investigation é distribuído no mesmo pacote do WPTV. Instalar o plugin já inclui o FIM Alert Investigation; a aba FORENSICS é registrada pelo mesmo `wptv.plugin.js` que registra a aba WPTV.
- **Log único:** todos os registros (requisições `/api/fim/*`, análise LLM, enriquecimento, erros de `_enrich_intel()`) vão para `/var/log/wazuh-process-tree/wptv.log`.
- **Sem serviço próprio:** o serviço systemd que gerencia o processo serve o WPTV e o FIM Alert Investigation via `server.py`.
- **Modelo:** Claude Sonnet 4.6, com `max_tokens` de 4096, instrução de escaping de barra invertida para caminhos Windows e tratamento de `JSONDecodeError` com log.

PRÉ-REQUISITO NGINX (ACHADO AO-4): O POST em `/api/fim/analyze` ultrapassa o limite padrão de 1 MB do nginx, que responde HTTP 413 como página HTML antes de repassar ao Flask; o frontend falha com `Unexpected token '<'` ao interpretar a resposta como JSON. Configure `client_max_body_size 10m;` no bloco `server {}` do arquivo nginx `wptv`.

6.6 Validações em Laboratório

Data	Agente	Cenário	Resultado
06/10/2026	labnix (008)	Alerta focal: `PROC_CONTEXT.txt` adicionado em `/home/kr/Downloads/`; 8 eventos de processo no contexto, com audit key `lpe_quartet_root_netlink_route`	`risk_score` 50, MEDIUM; T1068 com 52% de confiança; IOC `/usr/bin/vmtoolsd`; latência FIM-to-verdict de 16 s
08/10/2026	labnix (008)	Alerta 100303 (exclusão em massa), path `/var/tmp/mkinitramfs_prPy58/bin`; MITRE T1485, T1070, T1561	Análise entregue após ajuste do nginx (Achado AO-4)
08/10/2026	WZ-SRV	`_enrich_intel()` via script de teste, com contexto mock simulando beacon C2 e IP de teste, sem conexão direta a partir do lab	AbuseIPDB: score 100 e 7691 reports; VirusTotal not_found e OTX 404 (esperado para hash mock)
10/10/2026	labnix (008)	Regra 100310 no painel após incluir `100310` em `FIM_FILE_RULES`; arquivo `/tmp/fim_test_1791649577.txt`, evento added, modo realtime	Descrição completa com `[base: rule 554]` e campos `syscheck.*` corretos

Data	Agente	Cenário	Resultado
10/10/2026	LABDESK (004)	Simulação pós-exploração `up.ps1`, em três fases	44 alertas FIM; regra 100302 (nível 10); `risk_score` 87, CRITICAL; 11 técnicas MITRE; 9 IOCs

Tabela 17 - Validações do FIM Alert Investigation

Simulação pós-exploração em LABDESK (10/10/2026)

O script `up.ps1` executou três fases no agente LABDESK (004): **Fase 1, staging** (criação de `svc_host.exe`, `config.dat`, `update.log`, `payload_stage1.bin` e `readme_decrypt.txt` em AppData\Local\Temp, Downloads e Desktop); **Fase 2, reconhecimento** (PowerShell para cmd.exe, com `whoami`, `systeminfo`, `ipconfig`, `netstat`, `net`, `tasklist`, `arp` e `route`, em 11 comandos, com saída em `recon_output.txt`); **Fase 3, cópia lateral** (`svc_host.exe` copiado para `C:\Windows\Temp\` e Desktop). A correlação FIM com a árvore de processos (EventID 4688) foi confirmada no painel.

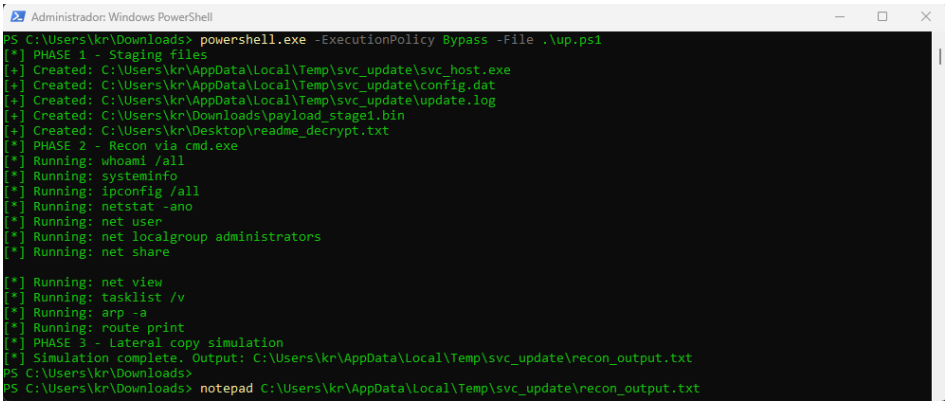


Figura 49 - Execução do script de simulação up.ps1 no LABDESK: fases de staging, reconhecimento e cópia lateral

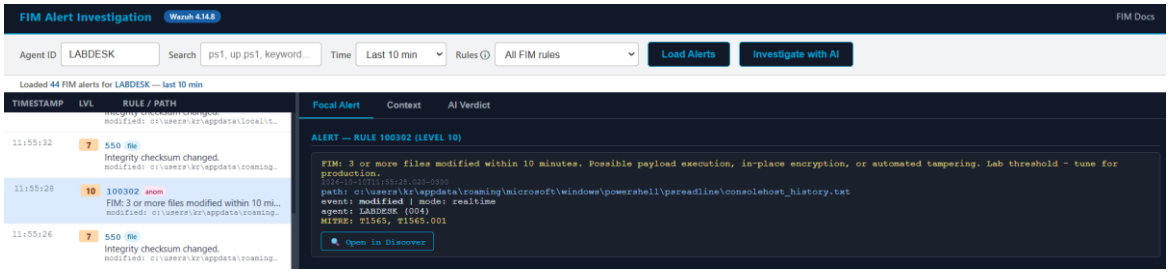


Figura 50 - Painel FIM Alert Investigation, aba Focal Alert: regra 100302 (nível 10), 44 alertas carregados para LABDESK nos últimos 10 min; MITRE T1565 e T1565.001

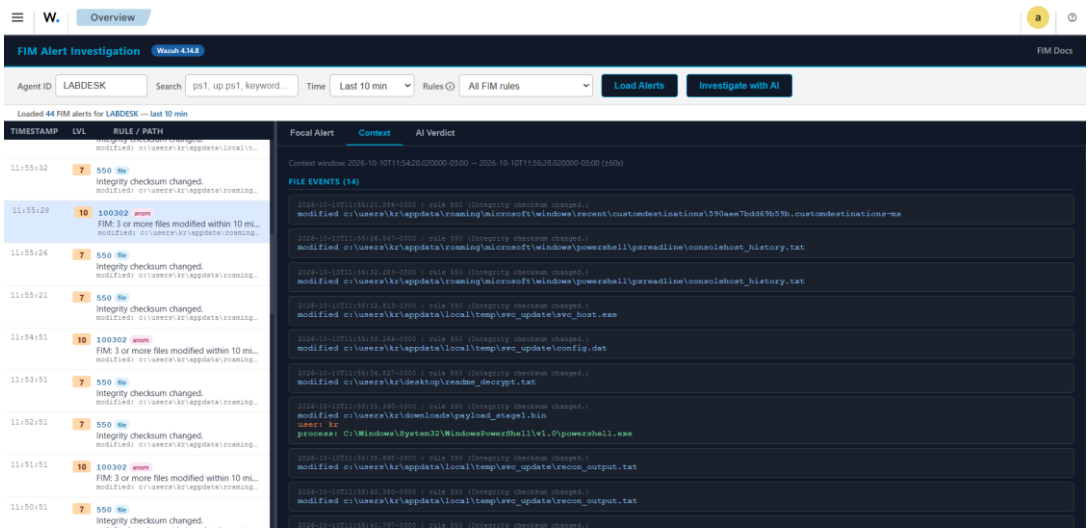


Figura 51 - Aba Context: 14 eventos de arquivo na janela de ±60 s do alerta 100302 selecionado

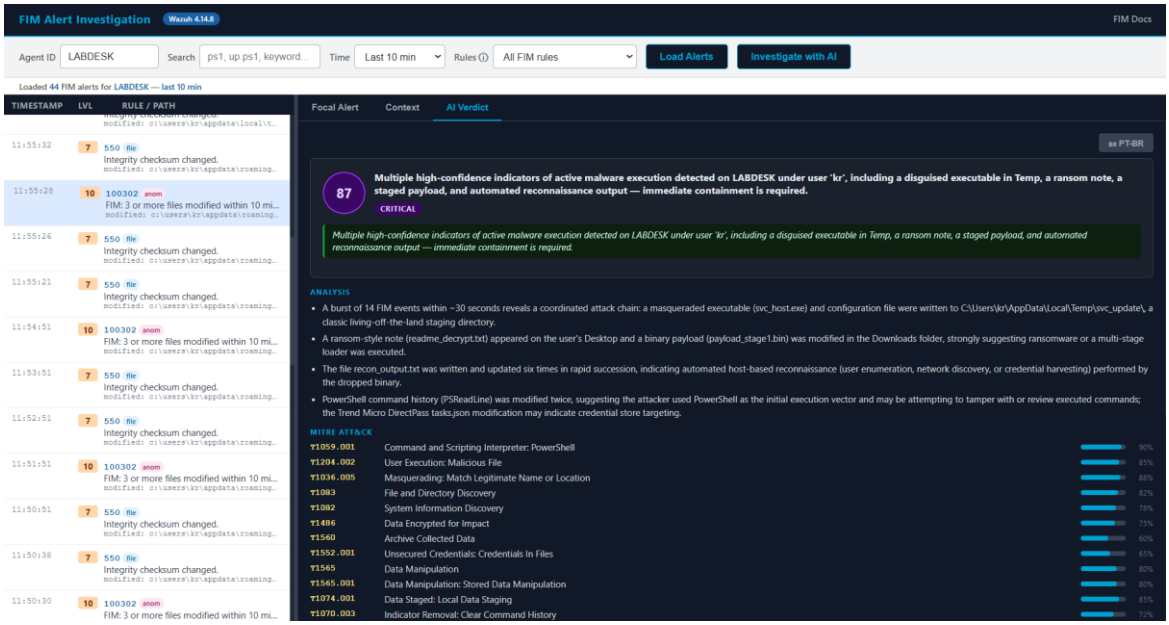


Figura 52 - Aba AI Verdict: risk_score 87, nível CRITICAL, com técnicas MITRE e percentuais de confiança

Técnica MITRE	Nome	Confiança
T1059.001	Command and Scripting Interpreter: PowerShell	90%
T1036.005	Masquerading: Match Legitimate Name or Location	88%
T1204.002	User Execution: Malicious File	85%
T1083	File and Directory Discovery	82%
T1565 e T1565.001	Data Manipulation e Stored Data Manipulation	80%
T1082	System Information Discovery	78%
T1486	Data Encrypted for Impact	75%
T1074.001	Data Staged: Local Data Staging	75%
T1070.003	Indicator Removal: Clear Command History	72%

Técnica MITRE	Nome	Confiança
T1552.001	Unsecured Credentials: Credentials In Files	65%
T1560	Archive Collected Data	60%

Tabela 18 - Técnicas MITRE ATT&CK no veredito da simulação up.ps1 (risk_score 87, CRITICAL)

O veredito identificou 9 IOCs: 2 hashes (SHA256 e MD5 do `ConsoleHost_History.txt` do PSReadLine) e 7 paths suspeitos, entre eles `svc_host.exe`, `recon_output.txt`, `readme_decrypt.txt`, `payload_stage1.bin`, `config.dat` e `tasks.json` de um gerenciador de senhas. As três ações recomendadas foram: isolamento imediato do host da rede, coleta forense com análise em sandbox e auditoria de eventos de autenticação com endurecimento por AppLocker ou WDAC.



Figura 53 - Aba AI Verdict: indicadores de comprometimento (hashes e paths) e ações recomendadas

> **Leitura do veredito:** O `risk_score` de 87 refere-se à atividade simulada de laboratório. Em produção, o veredito do LLM é um insumo de triagem que acelera a decisão do analista; ele não substitui a validação do alerta nem a decisão de contenção.

7. Regras Complementares do Ruleset Padrão

O arquivo `0015-ossec_rules.xml` que acompanha o Wazuh 4.14.x contém regras que não são de FIM propriamente ditas, mas determinam a eficácia do monitoramento de integridade em produção. Conhecê-las separa um deployment de FIM reativo de um resiliente.

Rule ID	Nível	Descrição	Relevância para o FIM	MITRE
504	3	Wazuh agent disconnected	Início de possível janela de cegueira	T1562.001
505	3	Wazuh agent removed	Agente removido: monitoramento encerrado	T1562.001
506	3	Wazuh agent stopped	Parada do serviço: FIM cessa por completo e em silêncio	T1562.001
560	7	FIM real-time queue is full. Some real-time events may be lost.	Fila inotify do kernel cheia: eventos realtime descartados	-
594 a 598, 750 a 752	variável	Registry key and value events	FIM de Registro do Windows (capítulo 3)	T1112 e correlatos

Tabela 19 - Regras nativas que condicionam a confiabilidade do FIM

7.1 Ciclo de Vida do Agente e Janelas de Cegueira (504, 505, 506)

As regras 504, 505 e 506 cobrem desconexão, remoção e parada do agente, todas mapeadas para o MITRE T1562.001 (Impair Defenses: Disable or Modify Tools). Quando o agente para, todo o monitoramento de integridade daquele endpoint cessa imediatamente e em silêncio: nenhum evento de arquivo é gerado, nenhum alerta chega ao servidor, e o endpoint apenas muda o ícone de status para disconnected.

```
<rule id="504" level="3">
  <if_sid>500</if_sid>
  <match>Agent disconnected</match>
  <description>Wazuh agent disconnected.</description>
  <mitre><id>T1562.001</id></mitre>
</rule>
<rule id="505" level="3">
  <if_sid>500</if_sid>
  <match>Agent removed</match>
  <description>Wazuh agent removed.</description>
  <mitre><id>T1562.001</id></mitre>
</rule>
<rule id="506" level="3">
  <if_sid>500</if_sid>
  <match>Agent stopped</match>
  <description>Wazuh agent stopped.</description>
  <mitre><id>T1562.001</id></mitre>
</rule>
```

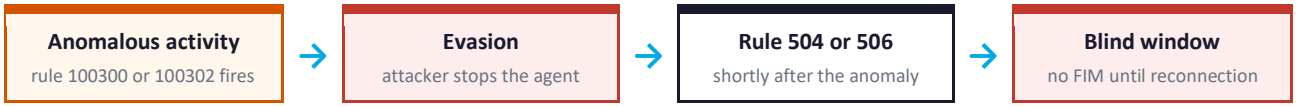


Figura 54 - Sequência de alta fidelidade: FIM anômalo seguido de desconexão ou parada do agente

O padrão de ameaça é a sequência 100300 ou 100302 seguida de 504 ou 506 em intervalo curto: um atacante que percebe o FIM ativo pode encerrar o agente antes de prosseguir. Correlacionar FIM anômalo imediatamente anterior a um evento de desconexão é um indicador comportamental de alta fidelidade, que vale implementar como regra customizada em ambientes de alta criticidade.

RECONEXÃO NÃO É RETROATIVA: Quando o agente reconecta após um período offline, o `syscheckd` executa novo scan de baseline e sincroniza o estado atual. Arquivos criados, modificados e excluídos durante o período offline não geram eventos retroativos. Qualquer lacuna no histórico FIM precedida por um evento 504 ou 506 deve ser investigada como possível janela de atividade não monitorada, e não descartada como ausência de atividade.

7.2 Overflow da Fila inotify em Tempo Real (560)

A regra 560 alerta quando a fila interna do inotify no kernel atingiu a capacidade máxima e eventos de tempo real estão sendo descartados. É o alerta nativo para o problema de limite de watches do capítulo 4. Nível 7, grupo syscheck, com descrição inequívoca: "Some real-time events may be lost".

```
<rule id="560" level="7">
  <if_sid>500</if_sid>
  <match>^ossec: Real-time inotify kernel queue is full.</match>
  <description>FIM real-time queue is full. Some real-time events may be lost.</description>
  <group>syscheck,agentless,pci_dss_11.5,...</group>
</rule>
```

A presença da 560 em um endpoint Linux é sinal direto de que `fs.inotify.max_user_watches` precisa ser aumentado ou de que o número de diretórios com realtime ou whodata precisa ser revisado. Em ambientes sob PCI DSS ou HIPAA, um alerta 560 é tecnicamente uma violação do controle de integridade: a ferramenta reporta explicitamente que não pode garantir a cobertura em tempo real.

O que a 560 não cobre: A regra 560 depende de a fila do kernel transbordar. Ela não indica se o realtime está entregando eventos ao manager. Para confirmar a entrega, consulte `archives.json` ou o índice `wazuh-archives-*` e verifique `syscheck.mode` igual a `realtime` (capítulo 4).

7.3 Registro do Windows

As regras 594, 597, 598, 750, 751 e 752 cobrem o FIM de Registro e estão detalhadas na seção 3.7. O ponto operacional relevante aqui é a correlação com o sistema de arquivos: persistência por chave Run gera um alerta 554 no arquivo depositado e um 752 no valor criado, correlacionáveis por agente e janela de tempo.

7.4 Active Response

O Active Response é a capacidade do Wazuh de executar ações automáticas nos endpoints em resposta a alertas. O detalhamento de configuração e de uso do Active Response não é o escopo deste handbook: incluí-lo aumentaria o número de páginas e poderia desviar o foco do estudo, dedicado ao FIM. Para quem deseja utilizá-lo, a referência é a documentação oficial do Wazuh, citada ao final desta seção.

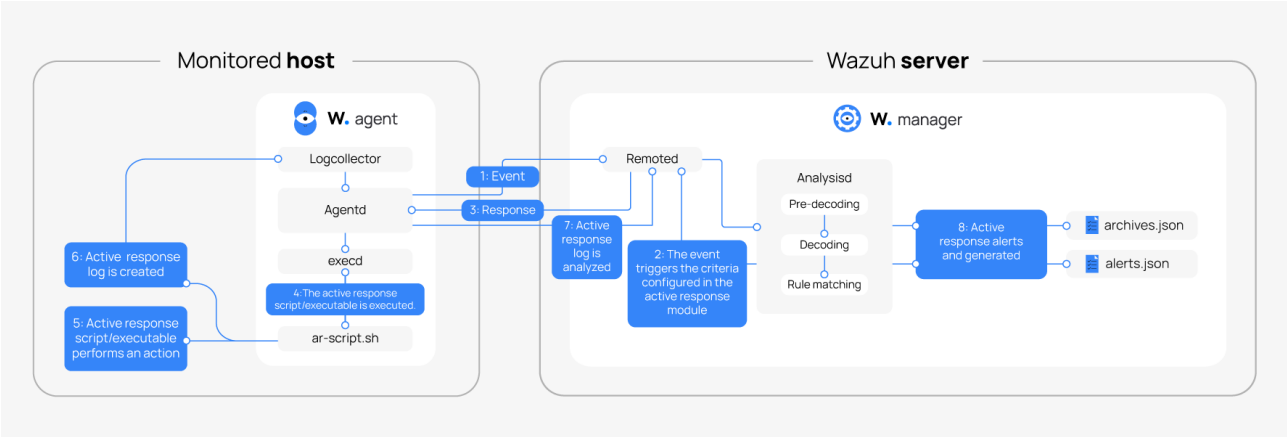


Figura 55 - Fluxo de trabalho do Active Response entre o agente Wazuh no host monitorado e o manager Wazuh no servidor Wazuh. Fonte: documentação do Wazuh, Active Response (https://documentation.wazuh.com/current/user-manual/capabilities/active-response/)

> **Types of active response:** "An active response can either be: Stateless, Stateful. Stateless active responses are one-time actions without an event definition to revert or stop them. Stateful responses revert or stop their actions after a period of time." (Wazuh documentation, Active Response)

Em resumo: respostas stateless são ações pontuais, sem uma definição de evento para revertê-las ou interrompê-las; respostas stateful revertem ou interrompem suas ações após um período de tempo.

Documentação oficial: https://documentation.wazuh.com/current/user-manual/capabilities/active-response/

8. FIM sob a Perspectiva do Adversário

Este capítulo aborda o FIM de uma perspectiva defensiva avançada: as técnicas que um atacante com acesso ao sistema pode usar para interferir no monitoramento de integridade e as contramedidas que o Wazuh oferece para detectar essa interferência. Entender como a detecção pode falhar é condição necessária para construir uma estratégia de monitoramento resiliente.

8.1 Técnicas de Evasão e Contramedidas

Técnica	Plataforma	Como interfere	Contramedida
Remoção das regras de auditoria (`auditctl -D`)	Linux	Root remove todas as regras carregadas, incluindo as do Wazuh; o whodata deixa de receber eventos a partir desse momento, o que cria janela de cegueira	Evento `CONFIG_CHANGE` no Audit gera a regra 80705; filtrar por esse ID e cruzar `audit.exe` com o processo responsável distingue a autoconfiguração legítima do Wazuh de outro processo
Reversão da política de auditoria (`auditpol /clear` ou PowerShell)	Windows	Reverte Sistema de Arquivos e Manipulação de Identificador	EventID 4719 (System audit policy was changed) com o GUID das duas subcategorias e sem o processo do Wazuh como originador indica interferência; o `windows_audit_interval` detecta a mudança e move os diretórios whodata para realtime
Parada ou remoção do agente	Ambas	Cessa todo o monitoramento do endpoint	Regras 504, 505 e 506 (T1562.001) e correlação com FIM anômalo imediatamente anterior
Timestomping	Ambas	Altera `mtime` para o passado para ocultar quando o arquivo foi criado ou modificado	`syscheck.mtime_after` muito anterior ao timestamp do alerta; mapeado ao T1070.006
Substituição atômica por `rename()`	Ambas	Arquivo malicioso criado fora do monitoramento substitui atomicamente um arquivo monitorado; o FIM vê modificação, e não criação	`report_changes` no diretório afetado captura o diff antes e depois, visível no campo `diff` do alerta

Tabela 20 - Técnicas de evasão contra o FIM e contramedidas disponíveis no Wazuh

8.2 Timestomping

No Linux, o `mtime` pode ser alterado diretamente por `touch -t` ou pela chamada de sistema `utimes`. O que não pode ser alterado a partir do userland é o `ctime`, o timestamp de mudança de metadados do inode, que o kernel atualiza automaticamente a cada alteração de metadado, inclusive do próprio `mtime`. Isso cria uma assinatura detectável: um evento de FIM com `mtime_after` no passado enquanto o alerta chega no presente indica um arquivo com `mtime` manipulado. O Wazuh captura `mtime_after` no evento, e a comparação com o timestamp do alerta no Discover revela o timestomping, um indicador de alta fidelidade. O mapeamento ao MITRE ATT&CK T1070.006 é deste handbook (a regra 550 nativa mapeia apenas T1565.001), e arquivos copiados com timestamps preservados (cp -p, tar, instalação de pacotes) também apresentam mtime_after antigo, por isso confirme com a atribuição de processo.

Uma consulta DQL no Discover expõe a anomalia:

```
rule.id:550 AND syscheck.mode:whodata
```

Depois de filtrar, ordene a coluna `syscheck.mtime_after` em ordem crescente: as datas mais antigas ficam no topo e são as candidatas. A confirmação é a comparação com o timestamp do alerta; uma diferença de dias

ou semanas em um arquivo de diretório crítico como `/usr/bin` ou `System32` merece investigação. Para atribuir, o campo `syscheck.audit.process.name` (Linux) ou `audit.process.name` (Windows) indica o executável responsável; `touch` no Linux ou `attrib` e PowerShell no Windows como autores de modificação em binário de sistema são um indicador adicional forte.

8.3 Padrão de Eventos de Ransomware

Com realtime ou whodata em diretórios críticos, o FIM produz um padrão reconhecível durante uma infecção por ransomware: a criptografia de arquivos existentes gera uma avalanche de eventos 550 em curto intervalo, no mesmo agente e frequentemente no mesmo diretório, muitas vezes com mudança de extensão em `syscheck.path` indicando a renomeação para a extensão do ransomware.



Figura 56 - Assinatura de ransomware no FIM: rajada de modificações, renomeações e escalação por correlação

O padrão da Figura 56 (rajada de modificações, renomeações e escalação por correlação) é coberto pelas regras validadas 100302 (3 ou mais eventos 550 em 10 minutos, nível 10) e 100304 (escalação por atividade mista, nível 14, que mapeia o T1486 - Data Encrypted for Impact). As definições, iguais às da seção 5.3, são reproduzidas abaixo.

```
<rule id="100302" level="10" frequency="3" timeframe="600" ignore="30">
  <if_matched_sid>550</if_matched_sid>
  <description>FIM: 3 or more files modified within 10 minutes. Possible payload execution,
in-place encryption, or automated tampering. Lab threshold - tune for
production.</description>
  <group>suspicious_modification,</group>
  <mitre><id>T1565</id><id>T1565.001</id></mitre>
</rule>

<rule id="100304" level="14" frequency="5" timeframe="600" ignore="30">
  <if_matched_group>syscheck_file</if_matched_group>
  <description>FIM: High volume mixed file activity - 5 or more create, modify, and delete
events within 10 minutes. Combined pattern suggesting active malware execution. Lab
threshold - tune for production.</description>
  <group>suspicious_combined,fim_escalation,</group>
  <mitre><id>T1485</id><id>T1486</id><id>T1565</id></mitre>
</rule>
```

CALIBRAÇÃO DOS LIMIARES: Os valores `frequency="3"` e `timeframe="600"` das regras 100302 e 100304 são valores de laboratório, como indicado no capítulo 5. Em ambientes com alta movimentação legítima de arquivos, como servidores de compilação e sistemas de backup, ajuste-os ao perfil de cada grupo de agentes antes do deploy. Sem whodata, as regras detectam o comportamento, mas não atribuem o processo responsável; com whodata, o alerta traz o processo responsável (seção 8.2).

8.4 Janelas de Cegueira Catalogadas

A tabela consolida as situações em que o FIM deixa de cobrir um endpoint sem que isso seja evidente, com a origem documentada neste handbook.

Janela de cegueira	Causa	Sinal observável	Referência
Agente parado ou desconectado	Evasão ou falha	Regras 504, 505, 506; status disconnected	7.1
Atividade durante período offline	Sincronização delta	Delta zero na reconexão; lacuna no histórico	2.7 e 7.1
Regras de auditoria removidas	`auditctl -D` ou falha de persistência	Regra 80705; `auditctl -l` vazio	4.3 e 4.4
Política de auditoria revertida	GPO ou `auditpol /clear`	EventID 4719	3.4 e 3.5
SACL não recuperada após restart do manager	whodata não restaurado após a reconexão; `windows_audit_interval` não reaplica a SACL	Ausência de EventID 4719 pós-reconexão; sem eventos whodata	AO-7
Whodata com fallback silencioso para realtime	Socket órfão ou auditd inacessível	Eventos sem atribuição de usuário	4.6 e 4.7
Fila inotify cheia	`max_user_watches` insuficiente	Regra 560	4.8 e 7.2
Alertas de criação ocultos pela regra proxy	Regra filha 100310 em nível 0 suprime o alerta da 554	`archives.json` com `syscheck.mode` realtime, evento `added` e `rule` nulo; ausência da 554 em `wazuh-alerts-*`	AC-7
Watches em arquivos individuais	Path com `/*` em `<directories>`	6016 por arquivo no `ossec.log`	AO-8

Tabela 21 - Janelas de cegueira do FIM e como reconhecê-las

8.5 Sysmon como Camada Complementar ao FIM

O FIM informa que um arquivo mudou, quais atributos mudaram e, em whodata, qual processo e qual usuário executaram a operação. O Sysmon observa a mesma atividade pelo lado do sistema operacional e registra eventos que o FIM não carrega, como a alteração da data de criação de um arquivo (EID 2), a criação ou sobrescrita de arquivos (EID 11) e a exclusão detectada (EID 26). Por isso o Sysmon é tratado neste handbook como uma camada complementar, que se soma ao FIM e não o substitui.

Para essa camada foi utilizado o repositório Unified Sysmon Configs, https://github.com/mym0us3r/Unified-Sysmon-Configs, como base da configuração do Sysmon nativo (schema 4.91) no endpoint Windows 11 do laboratório (LABDESK).

A partir desse repositório foram realizados testes com um novo conjunto de regras do Wazuh voltado à pesquisa, com cinco regras na faixa 100500 a 100599 (100521, 100540, 100541, 100554 e 100555). Cada regra Sysmon casa com um alerta FIM real da mesma operação de arquivo: criação, sobrescrita, exclusão, cópia de executável e alteração de datas. O conjunto foi disponibilizado no GitHub para que a comunidade possa testar, modificar e adaptar as regras à realidade do ambiente em que forem implementadas.

> **CONJUNTO DE REGRAS DE PESQUISA:** As regras Sysmon são um ponto de partida para testes e não integram o conjunto validado de seis regras FIM deste handbook. O caminho monitorado (`D:\File_Server_Test_Folder`), os níveis e a lista de interpretadores precisam ser ajustados ao ambiente antes de qualquer uso em produção.

Evidências no Wazuh Discover

As Figuras 57 a 68 reúnem as capturas reais do laboratório, na ordem em que as operações foram executadas no LABDESK em 12/10/2026. A Figura 57 mostra a aplicação da configuração do Sysmon. As demais são consultas

do Wazuh Discover, cada uma identificada pelo rule.id do alerta: as regras 100521, 100540, 100541, 100554 e 100555 são as regras Sysmon deste conjunto de pesquisa, e as regras 550, 553, 100300, 100302, 100303 e 100310 são as regras FIM do handbook. As Figuras 59, 61 e 62 documentam as operações em que o FIM é a única fonte: a sobrescrita com Set-Content, o append e a renomeação não geraram alerta Sysmon.

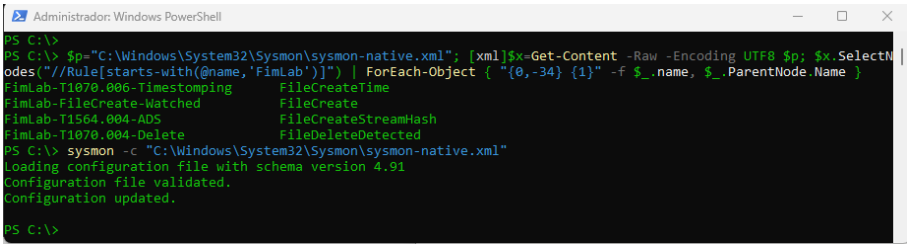


Figura 57 - Validação do XML e aplicação da configuração do Sysmon nativo com sysmon -c no LABDESK (schema 4.91)

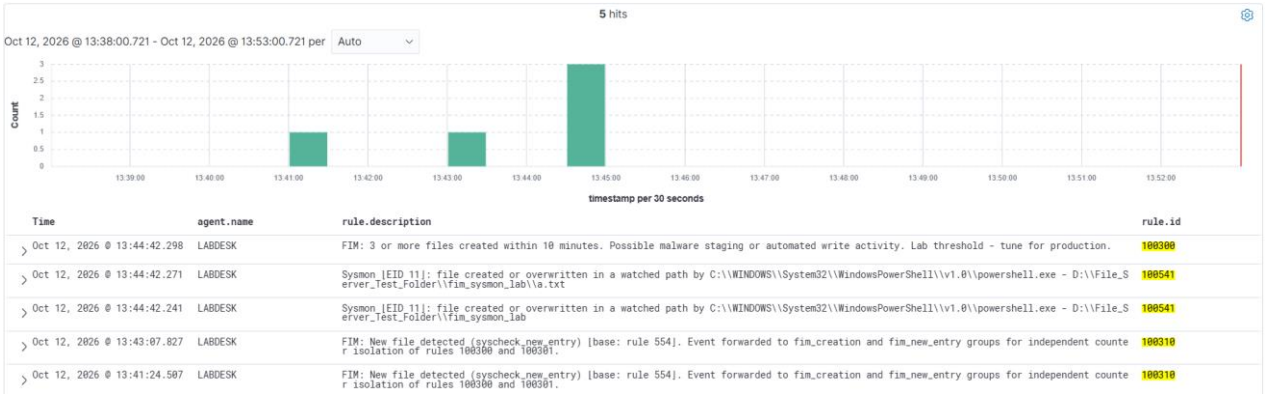


Figura 58 - Criação do a.txt no LABDESK: regra 100541 (Sysmon, EID 11, 13:44:42.241 e 13:44:42.271) seguida da 100300 (FIM, 13:44:42.298)

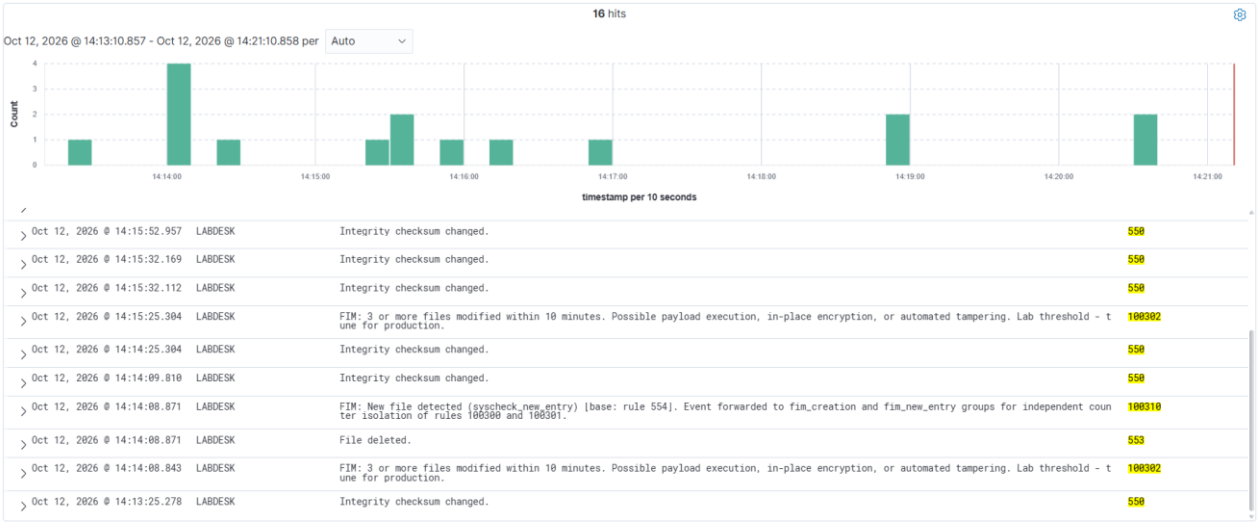


Figura 59 - Sobrescrita com Set-Content: alerta 550 (FIM) às 14:14:09.810, sem alerta Sysmon associado (intervalo de 14:13 a 14:21)



Figura 60 - Sobrescrita com redirecionamento do cmd.exe: regras 100555 (Sysmon, EID 26, 14:18:57.882) e 100541 (Sysmon, EID 11, 14:18:57.966) com a imagem cmd.exe

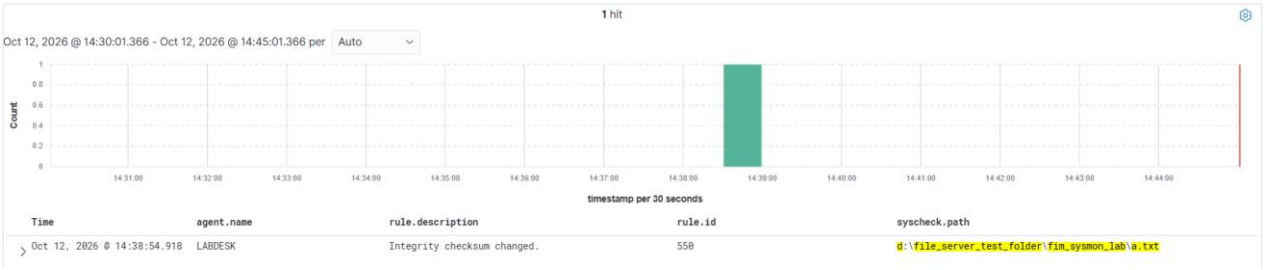


Figura 61 - Anexar linha ao a.txt: consulta combinada por caminho (FIM ou Sysmon) retorna somente o alerta 550 (FIM) de 14:38:54.918

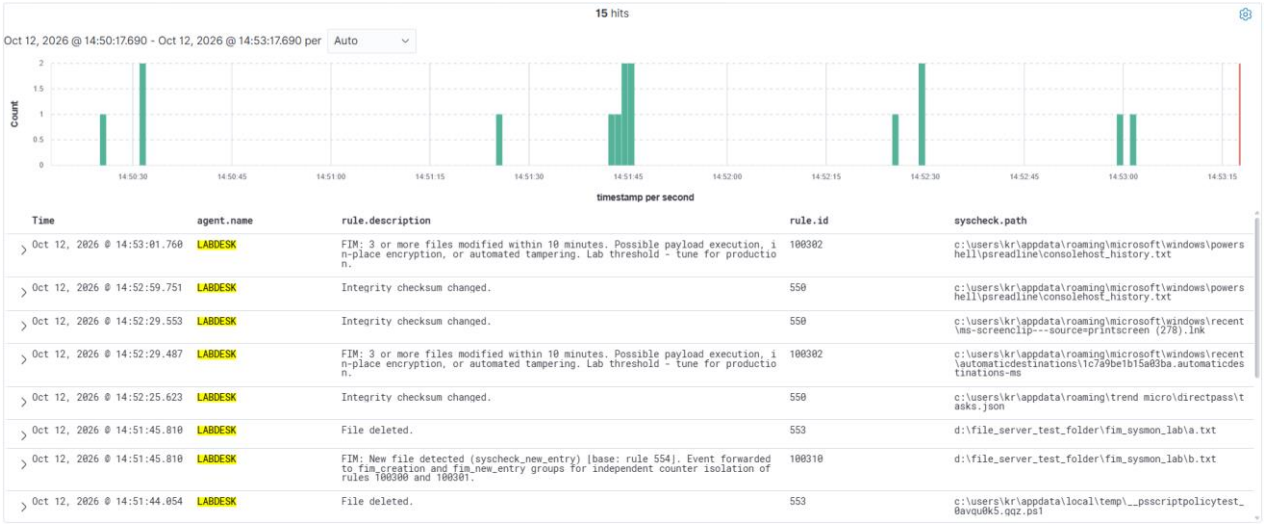


Figura 62 - Renomear arquivos: alertas 553 (a.txt) e 100310 (b.txt) do FIM às 14:51:45.810, sem alerta Sysmon associado



Figura 63 - Exclusão do b.txt: alerta 553 (FIM) e alerta 100555 (Sysmon, EID 26) no mesmo caminho



Figura 64 - Cópia do executável tool_ps.exe pelo PowerShell: alerta 100540 (Sysmon, EID 11)

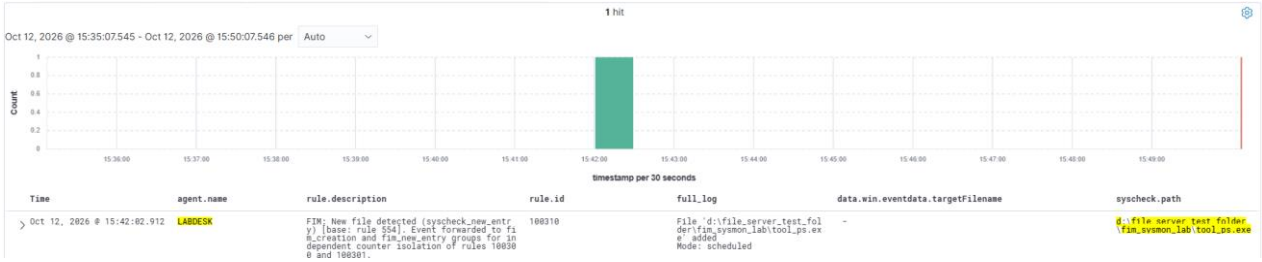


Figura 65 - Mesmo executável no FIM: alerta 100310 (proxy da 554, modo scheduled) com os hashes MD5 e SHA-256 do arquivo



Figura 66 - Alteração das datas do a.txt (timestomp): alerta 100521 (Sysmon, EID 2)

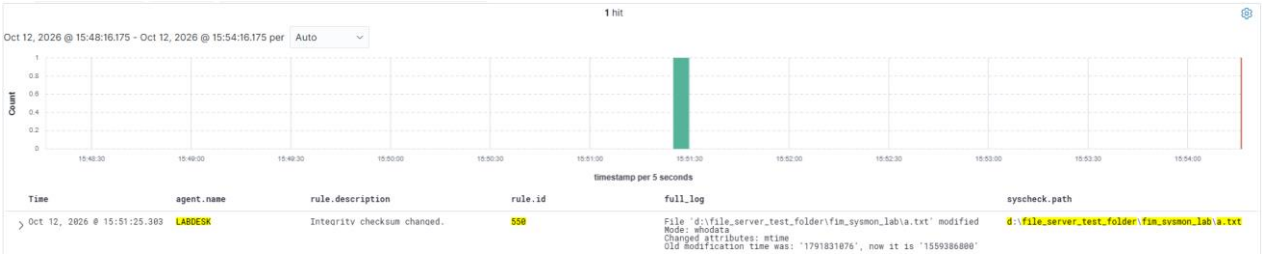


Figura 67 - Mesma operação no FIM: alerta 550 (modo whodata) com o atributo mtime alterado



Figura 68 - Limpeza dos arquivos de teste: 553 e 100555 (a.txt), 100554 (Sysmon, EID 26) e 100303 (FIM) para o tool_ps.exe

9. Compliance e MITRE ATT&CK

Uma das perguntas mais frequentes na comunidade Wazuh é como mapear a configuração de FIM para os requisitos de cada framework de compliance. O Wazuh etiqueta automaticamente cada alerta com os identificadores aplicáveis, visíveis nos campos `rule.pci_dss`, `rule.hipaa`, `rule.nist_800_53`, `rule.gdpr` e `rule.tsc`. Esta seção detalha o que cada requisito exige e como o FIM o atende.

Framework	Requisito	Atendimento pelo FIM	Campo no alerta
PCI DSS 4.0	11.5.2: mecanismo de detecção de mudanças em arquivos críticos, com comparação ao menos semanal	Scan agendado com `frequency` de até 604800 s atende ao mínimo; realtime ou whodata supera o requisito	`rule.pci_dss` (11.5 nas regras nativas)
PCI DSS 4.0	10.3.3: logs de auditoria copiados com segurança para servidor centralizado	Agentes enviam ao servidor, que envia ao indexer	`rule.pci_dss`
NIST 800-53 Rev 5	SI-7: verificação de integridade de software, firmware e informação	Checksums MD5, SHA-1 e SHA-256 (SI-7(6)); scan e tempo real (SI-7(1)); alertas automáticos (SI-7(2))	`rule.nist_800_53` (SI.7)
HIPAA	§164.312(c)(1) e (c)(2): proteção e autenticação de ePHI contra alteração ou destruição imprópria	FIM com whodata detecta modificação e registra usuário e processo	`rule.hipaa` (164.312.c.1 e c.2)
GDPR	Art. 5(1)(f): segurança adequada, incluindo proteção contra processamento não autorizado e perda ou destruição	FIM com whodata sobre diretórios com dados pessoais	`rule.gdpr` (II_5.1.f)
SOC 2 (TSC)	CC6.1, CC6.8, CC7.2 e CC7.3	Acesso lógico, prevenção de software malicioso, monitoramento de anomalias e avaliação de incidentes	`rule.tsc`

Tabela 22 - Mapeamento de requisitos de compliance para o FIM do Wazuh

9.1 PCI DSS 4.0

O requisito mais direto é o 11.5.2: um mecanismo de detecção de mudanças, como o monitoramento de integridade de arquivos, deve alertar o pessoal sobre modificações não autorizadas em arquivos críticos de sistema, de configuração ou de conteúdo, com escopo nos sistemas do CDE, e executar comparações de arquivos críticos pelo menos semanalmente. O requisito não especifica um formato de log, mas espera que os eventos de integridade sejam retidos e consultáveis, o que o índice `wazuh-alerts-*` com retenção configurada atende. O requisito 10.3.3, de cópia segura dos logs de auditoria para um servidor centralizado, é atendido estruturalmente pela arquitetura agente, servidor e indexer. Observe que as regras nativas do Wazuh etiquetam os alertas com a numeração anterior (por exemplo, pci_dss_11.5 nas regras 550, 553 e 554), de modo que o campo rule.pci_dss exibe 11.5 e não 11.5.2.

9.2 NIST 800-53 Rev 5

O controle central é o SI-7, que exige ferramentas de verificação de integridade para detectar modificações não autorizadas. O Wazuh FIM implementa o SI-7 com checksums MD5, SHA-1 e SHA-256, mencionados como proteção criptográfica no enhancement SI-7(6). O SI-7(1) exige verificações por ferramentas automatizadas, atendido pelo scan e pelo tempo real; o SI-7(2) exige notificação automática de violações, atendido pelo mecanismo de alertas; o SI-7(7) exige integração com o processo de resposta a incidentes. O campo `rule.nist_800_53` usa o identificador SI.7 em todos os alertas das regras 550, 553 e 554.

9.3 HIPAA

Os controles ficam em §164.312, Controles Técnicos. O §164.312(c)(1) exige políticas para proteger ePHI de alteração ou destruição imprópria; o §164.312(c)(2) exige um mecanismo para autenticar o ePHI, isto é, confirmar que não foi alterado ou destruído de forma não autorizada. Para ambientes de saúde, a configuração recomendada é `whodata` com `report_changes` nos diretórios de dados clínicos e nos arquivos de configuração de sistemas de prontuário eletrônico, de modo que qualquer modificação não autorizada produza alerta com atribuição completa de usuário e diff de conteúdo.

9.4 GDPR e TSC

O campo `rule.gdpr` usa o identificador II_5.1.f, correspondente ao Artigo 5(1)(f), que exige tratamento dos dados pessoais com segurança adequada, incluindo proteção contra processamento não autorizado ou ilegal e contra perda, destruição ou dano acidental. O FIM com whodata sobre diretórios com dados pessoais implementa esse princípio ao detectar e registrar qualquer acesso ou modificação não autorizada. O Wazuh também etiqueta alertas com os Trust Services Criteria CC6.1, CC6.8, CC7.2 e CC7.3.

> **Dica para auditorias:** O Wazuh Dashboard tem dashboards de compliance pré-construídos em Endpoint Security, que filtram e agrupam os alertas por framework. Eles exibem os eventos de FIM já mapeados para cada controle, o que reduz o esforço de preparação de evidências para auditores externos.

9.5 Mapeamento MITRE ATT&CK

As regras de correlação foram mapeadas para as técnicas MITRE ATT&CK relevantes, o que permite às equipes de resposta a incidentes correlacionar os alertas FIM com a cadeia de ataque do adversário.

Técnica	ID	Regras	Cenário de detecção
Data Manipulation	T1565	100300, 100302, 100304	Criação e modificação em massa de arquivos
Data Manipulation: Stored Data Manipulation	T1565.001	100300, 100302, 594, 750	Manipulação local de integridade de dados
Data Staged	T1074	100300	Preparo de arquivos para estágio ou exfiltração
Ingress Tool Transfer	T1105	100301	Propagação de binários por hash idêntico
Lateral Tool Transfer	T1570	100301	Replicação de ferramentas entre diretórios
Data Destruction	T1485	100303, 100304, 597, 751	Destruição massiva de arquivos e evidências
Indicator Removal	T1070	100303	Remoção de rastros e anti-forense
Disk Wipe	T1561	100303	Limpeza de disco pós-execução de malware
Data Encrypted for Impact	T1486	100304	Atividade mista sugerindo ransomware
Modify Registry	T1112	594, 597, 598, 750, 751, 752	Persistência e configuração via Registro do Windows

Técnica	ID	Regras	Cenário de detecção
Impair Defenses: Disable or Modify Tools	T1562.001	504, 505, 506	Parada, remoção ou desconexão do agente
Indicator Removal: Timestomp	T1070.006	550 (hunting por `mtime_after`)	Manipulação de timestamps (seção 8.2)

Tabela 23 - Mapeamento das regras de FIM para técnicas MITRE ATT&CK

10. Achados Técnicos Validados

Os achados abaixo foram identificados, investigados e tratados ao longo do desenvolvimento do pipeline, todos com evidências coletadas no ambiente de laboratório, sem suposições e sem simulação de saída; as entradas de teste, quando usadas, como o contexto mock da Tabela 17, estão identificadas como tal. Cada achado apresenta o comportamento observado, o impacto e a mitigação. Os IDs deste handbook unificam duas fontes que usam numerações próprias: **AC** (achados de correlação no analysisd, "LAB FINDINGS" do arquivo de regras) e **AO** (achados operacionais do handoff do pipeline). Os achados AC-1 a AC-7 e AO-1 a AO-8 são observações de laboratório na 4.14.x, não comportamentos declarados na documentação oficial da Wazuh.

ID	Achado	Tipo	Numeração original	Status
AC-1	same_field deve usar `md5_after`	Correlação	LAB FINDING [1]	RESOLVIDO
AC-2	`if_matched_sid` com múltiplos SIDs	Correlação	LAB FINDING [2]	RESOLVIDO
AC-3	Interferência de contador 100300 e 100301	Correlação	LAB FINDING [3]	RESOLVIDO
AC-4	`wazuh-logtest` não processa syscheck	Validação	LAB FINDING [4]	ESPERADO
AC-5	Interferência confirmada no Linux	Correlação	LAB FINDING [5]	RESOLVIDO
AC-6	Interferência filho-pai na 100300	Correlação	LAB FINDING [6]	RESOLVIDO
AC-7	Nível da 100310 e supressão da 554	Correlação	LAB FINDING [7]	RESOLVIDO
AO-1	`field name="file"` e não `syscheck.path`	Regras	Handoff, achado 1	RESOLVIDO
AO-2	Saturação por fontes de ruído	Endpoint	Handoff, achado 2	RESOLVIDO
AO-3	`auditpol` em workgroup Windows	Endpoint	Handoff, achado 3	RESOLVIDO
AO-4	`client_max_body_size` do nginx	Integração	Handoff, achado 4	RESOLVIDO
AO-5	AbuseIPDB via `connect` do auditd	Integração	Handoff, achado 5	RESOLVIDO
AO-6	VirusTotal e payloads customizados	Integração	Handoff, achado 6	ESPERADO
AO-7	SACL não recuperada após restart do manager	Windows	Handoff, achado 7	MITIGADO
AO-8	Path com `/*` expande o glob	Endpoint	Handoff, achado 8	RESOLVIDO

Tabela 24 - Índice dos 15 achados técnicos

10.1 Correlação no analysisd (AC)

AC-1 Mapeamento correto do campo same_field

RESOLVIDO Correlação (analysisd)

Observação	Em regras de correlação FIM, `same_field` deve referenciar o nome do campo decodificado pelo decoder do analysisd. Para eventos FIM o campo correto é `md5_after`, e não `md5`.
Evidência	Comparação entre o JSON do alerta em `wazuh-alerts-*` e a saída do decoder interno do analysisd. No laboratório (4.14.x), `same_field` com `md5` não correlaciona.
Impacto	Com `md5`, a regra 100301 não correlaciona: o campo não é encontrado no evento e o acumulador não avança.
Mitigação	`<same_field>md5_after</same_field>` na regra 100301.

Origem	LAB FINDING [1] de `fim_anomaly_rules.xml`

AC-2 if_matched_sid com múltiplos valores não é confiável

RESOLVIDO Correlação (analysisd)

Observação	A sintaxe de SIDs separados por vírgula em `if_matched_sid` (por exemplo, `550,553,554`) não é suportada de forma confiável em regras de correlação temporal na 4.14.x.
Mitigação	Usar `if_matched_group` com um grupo compartilhado. A regra 100304 usa `syscheck_file`, grupo nativo das regras 550, 553 e 554, confirmado em `0015-ossec_rules.xml`.
Origem	LAB FINDING [2]

AC-3 Interferência de contador entre as regras 100300 e 100301

RESOLVIDO Correlação (analysisd)

Observação	Com `if_matched_sid` 554 em ambas, o analysisd compartilha o acumulador de eventos: o disparo da 100300 no terceiro evento consome o estado do contador e impede a 100301 de acionar corretamente.
Mitigação	Regra auxiliar 100310 (filha de 554) reencaminha cada evento 554 para os grupos `fim_creation` (100300) e `fim_new_entry` (100301), criando dois acumuladores independentes.
Origem	LAB FINDING [3], identificado inicialmente no LABDESK (Windows)

AC-4 wazuh-logtest não processa eventos syscheck

ESPERADO Ferramenta de validação

Observação	O `wazuh-logtest` não processa eventos syscheck pelas fases 2 e 3. Os decoders `syscheck_new_entry`, `syscheck_integrity_changed` e `syscheck_deleted` são internos ao pipeline do analysisd e indisponíveis no logtest.
Impacto	Regras FIM só podem ser validadas pelo pipeline real, com agente gerando eventos. Cada ciclo de teste exige deploy das regras, reinício do `wazuh-manager` e geração de eventos reais nos endpoints.
Origem	LAB FINDING [4]

AC-5 Interferência de contador confirmada no Linux

RESOLVIDO Correlação (analysisd)

Observação	O AC-3 foi identificado no Windows (LABDESK). Testes em labnix (09/10/2026) confirmaram o mesmo comportamento no Linux quando 100300 e 100301 compartilham `if_matched_sid` 554. A interferência é característica do analysisd no servidor, independente do sistema operacional do agente.
Mitigação	O padrão proxy da 100310 com grupos separados é eficaz e validado nas duas plataformas.
Origem	LAB FINDING [5]

AC-6 Interferência filho-pai na regra 100300

RESOLVIDO Correlação (analysisd)

Evidência	A 100303 (`if_matched_sid` 553, sem regra filha em 553) dispara em todos os testes; a 100300 (`if_matched_sid` 554, com a filha 100310 em 554) não disparava. O evento da regra filha, inserido entre eventos 554 consecutivos, desorganizava o acumulador.
Mitigação	A 100310 emite para `fim_creation`, e a 100300 passou de `if_matched_sid` 554 para `if_matched_group` `fim_creation`, isolando por completo seu acumulador.
Origem	LAB FINDING [6]

AC-7 Nível da regra 100310 e supressão da regra 554

RESOLVIDO Correlação (analysisd)

Observação	Com a filha 100310 (`if_sid` 554) em nível maior que zero, o analysisd suprime o alerta da regra pai 554: apenas a 100310 chega a `wazuh-alerts-*`. A tentativa de nível 0 suprimiu o alerta por completo, restando o evento só em `wazuh-archives-*`, sem campo `rule`.
Evidência	Varredura do `alerts.json`: mais de 40 entradas com `rule.id=100310` e zero com `rule.id=554`. Validado em 10/10/2026: LABDESK às 02:14:29, labnix às 02:30:14 e 13:26:17.
Mitigação	100310 mantida em nível 5; `100310` incluído em `FIM_FILE_RULES` do `logic_fim.py`; descrição com `[base: rule 554]` para referência do analista.
Origem	LAB FINDING [7] do arquivo de regras (supressão pai e filha)

10.2 Operação e Integração (AO)

AO-1 Campo field name="file", e não syscheck.path

RESOLVIDO Configuração de regras

Observação	Em regras FIM customizadas, o campo correto para correspondência de path é `<field name="file">`. O campo `syscheck.path` não é reconhecido pelo analysisd.
Impacto	Falha silenciosa: o analysisd não reporta erro, a regra simplesmente não casa.

AO-2 Saturação do syscheck por fontes de ruído

RESOLVIDO Configuração de endpoint

Observação	Chrome (`.crdownload`), Trend Micro DirectPass, Sublime Text e MobaXterm gravam com alta frequência nos diretórios monitorados. Sem filtro, o volume de eventos supera o detectável útil.
Mitigação	`<ignore type="sregex">` no `ossec.conf` do agente, cobrindo os paths específicos de cada fonte de ruído.

AO-3 auditpol obrigatório para whodata em workgroup Windows

RESOLVIDO Configuração de endpoint

Observação	No LABDESK, em ambiente workgroup sem Group Policy, a auditoria de acesso a objetos em "Êxito" precisou ser habilitada manualmente. Sem ela, a SACL configurada pelo Wazuh é invalidada silenciosamente; o Wazuh emite o erro 6021 e o whodata não coleta dados de usuário nem de processo.

Mitigação	Habilitar as duas subcategorias:
	`auditpol /set /subcategory:"File System" /success:enable`
	`auditpol /set /subcategory:"Handle Manipulation" /success:enable`

Nota	A autoconfiguração do Wazuh é suficiente em condições normais (seção 3.2); este achado registra o comportamento observado no LABDESK. Use GUIDs se o idioma do Windows não for inglês.

AO-4 client_max_body_size do nginx para payloads de análise LLM

RESOLVIDO Integração (nginx e Flask)

Observação	O POST em `/api/fim/analyze` (contexto focal, `file_events` e `process_events`) ultrapassa o limite padrão de 1 MB do nginx, que devolve HTTP 413 como página HTML antes de repassar ao Flask.
Impacto	O frontend falha com `Unexpected token '<'` ao executar `JSON.parse` na resposta.
Mitigação	`client_max_body_size 10m;` no bloco `server {}` do arquivo nginx `wptv`. Validado em 08/10/2026.

AO-5 AbuseIPDB só é consultado via connect syscall do auditd

RESOLVIDO Integração (enriquecimento)

Observação	O pipeline extrai IPs de `data.audit.addr` nos eventos de processo. IPs presentes no conteúdo de arquivos monitorados não são extraídos.
Mitigação	Regras de auditd para o syscall `connect` em labnix (`uid>=1000` e `uid=0`), carregadas por `augenrules --load` e confirmadas por `auditctl -l` (seção 6.4).

AO-6 VirusTotal não detecta payloads customizados

ESPERADO Integração (enriquecimento)

Observação	O VirusTotal retornou 0 detecções para o `beacon_test.sh` criado no laboratório (SHA256 iniciado em `5ca772c3`). Scripts customizados ou inéditos não têm assinatura prévia.
Leitura	O VirusTotal é sinal secundário. Para payloads novos, o AbuseIPDB via `connect` syscall é o sinal primário.

AO-7 SACL não restaurada após restart do manager

MITIGADO Plataforma Windows

Cenário	Manager reiniciado às 01:47:38 (10/10/2026). O agente LABDESK perde a conexão; com `auto_restart=yes` e `time-reconnect=60`, o processo WazuhSvc reinicia e reconecta. Depois da reconexão, o whodata em `D:\File_Server_Test_Folder` não gerou eventos syscheck.
Linha do tempo	01:47:38 Restart do manager 02:10:35 Arquivo criado em `D:\File_Server_Test_Folder`, sem evento syscheck no `archives.log`

	02:13:04 EventID 4719 (File System e Handle Manipulation, "Success added", conta LABDESK$ S-1-5-18): SACL reaplicada somente após `Restart-Service WazuhSvc` manual 02:14:29 Arquivo no Desktop (realtime) capturado normalmente
Causa raiz	`windows_audit_interval=60` verifica a política de auditoria e as SACLs enquanto o motor whodata está em execução e, em caso de divergência, move os diretórios para realtime (documentação oficial); não reaplica a SACL. No laboratório, a SACL só foi reaplicada pelo restart do agente (EventID 4719 às 02:13:04). O restart do agente na reconexão foi inferido no laboratório e não é documentado pela Wazuh. Gap total de cerca de 25 minutos sem cobertura whodata.
Mitigação	Após restart do manager, reiniciar o agente no endpoint Windows afetado. Para automação, considerar um script de monitoramento que detecte a ausência de EventID 4719 pós-reconexão e reinicie o serviço.
Nota	O Desktop (realtime) não foi afetado: `ReadDirectoryChangesW` não depende de SACL e não é interrompido pela reconexão.

AO-8 Path com /* expande o glob em arquivos individuais

RESOLVIDO Configuração de endpoint

Cenário	Paths `/home/kr/Downloads/*` e `/home/kr/Desktop/*` configurados temporariamente em labnix.

Evidência com /*	ossec.log às 01:35:57:
	`(6016): Directory set for real time monitoring: '/home/kr/Downloads/file.zip'`
	`(6016): Directory set for real time monitoring: '/home/kr/Downloads/pewpewpew.txt'`
	O `syscheckd` expandiu o glob e registrou watches em arquivos existentes; novos arquivos não tinham watch e o inotify não os capturava.

Evidência sem /*	ossec.log às 02:28:18:
	`(6003): Monitoring path: '/home/kr/Downloads', with options '... | realtime'`
	`(6016): Directory set for real time monitoring: '/home/kr/Downloads'`
	Watch registrado no diretório; novos arquivos capturados corretamente.

Mitigação	Remover `/*` dos paths de diretório em `<directories>`.

11. Configurações de Referência

11.1 Wazuh 4.14.8: Atualizações do FIM

A versão 4.14.8, lançada em 23 de setembro de 2026, trouxe melhorias e correções para o módulo FIM, em especial para o provider whodata via eBPF no Linux. As alterações afetam a qualidade dos alertas de integridade, a precisão dos campos de auditoria e a estabilidade do pipeline descrito neste handbook.

Item	Tipo	Mudança	Impacto
#32467	Adição	Campos `audit_uid`, `audit_name`, `audit_gid` e `audit_group_name` nos alertas FIM whodata do provider eBPF	Identidade do usuário e do grupo sem resolver UID manualmente; campos verificáveis em `syscheck.audit.*` no Discover
#39358	Correção	Inversão entre `user_id` e `group_id` nos alertas FIM whodata do provider eBPF	Regras que filtram por UID específico passam a receber o valor correto
#39335	Correção	Atribuição de todas as modificações ao usuário root quando o kernel não reportava `loginuid`	O eBPF trata a ausência de `loginuid`, evitando atribuição errada a root
#38145	Correção	Escrita fora dos limites do buffer heap no `wazuh-analysisd` na geração de alertas FIM; o buffer `full_log` passou a ser redimensionado antes da escrita	Elimina risco de corrupção de memória com payloads extensos; relevante em FIM de alta volumetria

Tabela 25 - Mudanças da 4.14.8 relevantes para o FIM (release notes oficiais)

11.2 ossec.conf Endurecido: Linux (labnix)

Trecho do bloco `syscheck` validado em labnix (Ubuntu 24.04.5, kernel 6.17.0, agente 4.14.8). O endereço do servidor é um placeholder.

```
<client>
  <server>
    <address>YOUR-WAZUH-SERVER-IP</address>
    <port>1514</port>
    <protocol>tcp</protocol>
  </server>
  <notify_time>20</notify_time>
  <time-reconnect>60</time-reconnect>
  <auto_restart>yes</auto_restart>
</client>

<syscheck>
  <disabled>no</disabled>
  <frequency>60</frequency>
  <scan_on_start>yes</scan_on_start>

  <directories check_all="yes" whodata="yes">/etc</directories>
  <directories check_all="yes" whodata="yes">/usr/bin,/usr/sbin</directories>
  <directories check_all="yes" whodata="yes">/bin,/sbin,/boot</directories>

  <directories check_all="yes" realtime="yes">/home/kr/Downloads</directories>
  <directories check_all="yes" realtime="yes">/home/kr/Desktop</directories>
  <directories check_all="yes" realtime="yes">/tmp</directories>
  <directories check_all="yes" realtime="yes">/var/tmp</directories>

  <ignore type="sregex">.log$|.swp$</ignore>
  <nodiff>/etc/ssl/private.key</nodiff>
  <skip_nfs>yes</skip_nfs>
  <skip_dev>yes</skip_dev>
  <skip_proc>yes</skip_proc>
  <skip_sys>yes</skip_sys>
```

```
  <process_priority>10</process_priority>
  <max_eps>50</max_eps>

  <synchronization>
    <enabled>yes</enabled>
    <interval>5m</interval>
    <max_eps>10</max_eps>
  </synchronization>
</syscheck>

<localfile>
  <log_format>audit</log_format>
  <location>/var/log/audit/audit.log</location>
</localfile>
```

> **Notas:** Os paths de diretório são declarados sem `/*` (Achado AO-8). O `frequency` de 60 s é valor de laboratório, usado para acelerar a validação dos cenários; em produção, o padrão é 43200 s. Os valores de synchronization exibidos são os padrões do Wazuh.
>
> A regra de auditd para o syscheck cobre `connect` em `/etc/audit/rules.d/net_connect.rules` (seção 6.4).

11.3 ossec.conf Endurecido: Windows (LABDESK)

Trecho do bloco `syscheck` validado no LABDESK (Windows 11 Pro, agente 4.14.8), mantendo os caminhos padrão do Wazuh e adicionando os caminhos de staging de malware e de persistência. Os valores de synchronization exibidos são os padrões do Wazuh.

```
<syscheck>
  <disabled>no</disabled>
  <directories realtime="yes">%PROGRAMDATA%\Microsoft\Windows\Start
Menu\Programs\Startup</directories>

  <directories check_all="yes" realtime="yes">C:\Users\kr\Downloads</directories>
  <directories check_all="yes" realtime="yes"
report_changes="yes">C:\Users\kr\AppData\Local\Temp</directories>
  <directories check_all="yes" realtime="yes"
report_changes="no">C:\Users\kr\AppData\Roaming</directories>
  <directories check_all="yes" realtime="yes"
report_changes="yes">C:\Users\kr\Desktop</directories>
  <directories check_all="yes" realtime="yes" report_changes="yes">C:\Windows\Temp</directories>

  <directories check_all="yes" whodata="yes"
report_changes="no">C:\Windows\System32</directories>
  <directories check_all="yes" whodata="yes"
report_changes="no">C:\Windows\SysWOW64</directories>
  <directories check_all="yes" whodata="yes"
report_changes="no">C:\Windows\System32\drivers</directories>
  <directories check_all="yes" whodata="yes"
report_changes="no">C:\Windows\System32\wbem</directories>
  <directories check_all="yes" whodata="yes"
report_changes="no">C:\Windows\System32\WindowsPowerShell</directories>

  <directories check_all="yes" whodata="yes"
report_changes="yes">D:\File_Server_Test_Folder</directories>

  <windows_registry
arch="both">HKEY_LOCAL_MACHINE\Software\Microsoft\Windows\CurrentVersion\Run</windows_registry>
  <windows_registry
arch="both">HKEY_CURRENT_USER\Software\Microsoft\Windows\CurrentVersion\Run</windows_registry>

  <ignore type="sregex">.etl$|.tmp$|.~$</ignore>
  <windows_audit_interval>60</windows_audit_interval>
  <process_priority>10</process_priority>
  <max_eps>50</max_eps>
  <synchronization>
```

```
    <enabled>yes</enabled>
    <interval>5m</interval>
    <max_eps>10</max_eps>
  </synchronization>
</syscheck>
```

11.4 Estado do Servidor (WZ-SRV)

Arquivo	Caminho	Estado
`fim_anomaly_rules.xml`	`/var/ossec/etc/rules/`	6 regras (100310 e 100300 a 100304); 100310 em nível 5; permissões 660 wazuh:wazuh
`fim_sysmon_rules.xml`	`/var/ossec/etc/rules/`	5 regras (100521, 100540, 100541, 100554 e 100555); permissões 660 wazuh:wazuh
`ossec.conf` (manager)	`/var/ossec/etc/`	`logall_json` yes; filebeat archives ativo; syscheck `frequency` 60
`logic_fim.py`	`/usr/share/wazuh-dashboard/plugins/process_tree_api/`	`FIM_FILE_RULES` com 100310; `_enrich_intel()` com VirusTotal, OTX e AbuseIPDB; `max_tokens` 4096; handler de `JSONDecodeError`
`server.py`	`process_tree_api/`	Rotas `/api/fim/*`
`fim.html`	`process_tree_api/public/`	Painel com ordenação decrescente, busca por hostname, busca livre e divisor redimensionável
`wptv` (nginx)	`/etc/nginx/sites-available/`	`client_max_body_size 10m`

Tabela 26 - Estado dos arquivos do laboratório no servidor

11.5 Conclusão

O FIM (File Integrity Monitoring, ou Monitoramento de Integridade de Arquivos) não é um produto isolado. É um recurso nativo do agente Wazuh, integrado a um ecossistema que coleta, decodifica, correlaciona, indexa e visualiza dados de segurança. Um único agente monitora arquivos e o Registro do Windows, identifica quem realizou uma alteração por meio do Whodata, associa eventos a requisitos de conformidade e encaminha os dados para os fluxos de SIEM e XDR, tudo em uma plataforma aberta e gratuita.

O valor do FIM, entretanto, depende do planejamento, da configuração e da validação, e não apenas de sua ativação. As 15 constatações apresentadas no Capítulo 10 demonstram que a cobertura do monitoramento pode desaparecer sem que isso gere erros evidentes nos logs (AO-7 e AO-8). Também mostram que a causa frequentemente está em uma camada abaixo do agente, como uma política de auditoria do sistema operacional ou um objeto de Política de Grupo (GPO) esquecido. Essas constatações são observações de laboratório realizadas na série 4.14.x, cada uma documentada com evidências e medidas de mitigação.

Uma implementação bem configurada de FIM começa pela seleção criteriosa dos caminhos a serem monitorados, priorizando áreas de preparação de malware, locais associados a mecanismos de persistência e binários do sistema. Também envolve o uso do Whodata quando a identificação da origem das alterações é importante, o controle de ruído por meio de diretivas ignore, o ajuste das regras de correlação com base em uma linha de base conhecida e a verificação contínua da cobertura, incluindo regras relacionadas ao ciclo de vida dos agentes e a regra 80705. A partir dessa estrutura, a correlação de eventos, o Sysmon como camada complementar de telemetria e a investigação assistida por IA ajudam a reduzir o tempo entre a geração de um alerta e a compreensão do que aconteceu, mantendo sempre a decisão final sob responsabilidade do analista. No laboratório, a latência medida entre o alerta e o veredito foi de 16 segundos.

Para organizações de qualquer porte, os princípios são os mesmos, embora as prioridades variem. Em ambientes menores, o monitoramento de alguns diretórios críticos, combinado com as configurações padrão do Wazuh e o mapeamento de conformidade, já pode proporcionar visibilidade e evidências úteis para auditorias. Em ambientes médios e grandes, os grupos de agentes e a configuração centralizada permitem definir perfis específicos por função, como servidores de arquivos, servidores de compilação e servidores de backup, cada um com ajustes próprios para o volume de eventos e os limiares de monitoramento. Os valores apresentados neste Handbook foram obtidos em um laboratório com 3,9 GB de RAM e não constituem uma referência para dimensionamento de ambientes de produção. Portanto, cada ambiente precisa estabelecer e validar sua própria linha de base.

O FIM não é uma solução que se instala e depois se esquece. É um controle de segurança que exige planejamento, validação e manutenção contínua. Quando esses requisitos são atendidos, ele se torna uma das formas mais acessíveis de transformar a integridade de arquivos em sinais relevantes para detecção, resposta a incidentes e conformidade, ao alcance de equipes de diferentes portes que adotam o Wazuh.

Espero que este guia avançado contribua de forma significativa para toda a comunidade Wazuh, apoiando usuários, analistas e profissionais de segurança com diferentes níveis de conhecimento e experiência. Meu objetivo é compartilhar conhecimento prático, pesquisas validadas, evidências obtidas em laboratório e informações técnicas que ajudem os leitores a aprofundar sua compreensão sobre o Monitoramento de Integridade de Arquivos (FIM), fortalecer suas capacidades de monitoramento de segurança, aprimorar a detecção e a investigação de incidentes e utilizar o Wazuh de maneira mais eficaz em seus próprios ambientes.

Além de servir como referência técnica, espero que este trabalho incentive a curiosidade, o pensamento crítico, o compartilhamento de conhecimento e o aprendizado contínuo. Seja você alguém que está dando os primeiros passos com o Wazuh ou um profissional com anos de experiência em cibersegurança, espero que este guia ofereça informações valiosas, orientações práticas e ideias que inspirem novas pesquisas e contribuam para uma comunidade de segurança mais forte, mais bem informada e colaborativa.

12. Referências Oficiais

Os links de documentação e referência abaixo foram reabertos e confirmados em 10 e 12 de outubro de 2026. A última linha é o repositório deste projeto.

Reference	Address	Status
Release notes 4.14.8	https://documentation.wazuh.com/current/release-notes/release-4-14-8.html	**Verificado**
Custom FIM rules	https://documentation.wazuh.com/current/user-manual/capabilities/file-integrity/creating-custom-fim-rules.html	**Verificado**
Malware persistence detection	https://documentation.wazuh.com/current/user-manual/capabilities/file-integrity/use-cases/detecting-malware-persistence-technique.html	**Verificado**
FIM: overview	https://documentation.wazuh.com/current/user-manual/capabilities/file-integrity/index.html	**Verificado**
FIM: how it works	https://documentation.wazuh.com/current/user-manual/capabilities/file-integrity/how-it-works.html	**Verificado**
FIM: basic settings	https://documentation.wazuh.com/current/user-manual/capabilities/file-integrity/basic-settings.html	**Verificado**
FIM: advanced settings	https://documentation.wazuh.com/current/user-manual/capabilities/file-integrity/advanced-settings.html	**Verificado**

Windows Registry monitoring	https://documentation.wazuh.com/current/user-manual/capabilities/file-integrity/windows-registry-monitoring.html	**Verificado**
ossec.conf reference: syscheck	https://documentation.wazuh.com/current/user-manual/reference/ossec-conf/syscheck.html	**Verificado**
System calls monitoring	https://documentation.wazuh.com/current/user-manual/capabilities/system-calls-monitoring/how-it-works.html	**Verificado**
Audit configuration	https://documentation.wazuh.com/current/user-manual/capabilities/system-calls-monitoring/audit-configuration.html	**Verificado**
Centralized configuration	https://documentation.wazuh.com/current/user-manual/reference/centralized-configuration.html	**Verificado**
PCI DSS compliance and FIM	https://documentation.wazuh.com/current/compliance/pci-dss/file-integrity-monitoring.html	**Verificado**
HIPAA compliance and FIM	https://documentation.wazuh.com/current/compliance/hipaa/file-integrity-monitoring.html	**Verificado**
NIST compliance and FIM	https://documentation.wazuh.com/current/compliance/nist/file-integrity-monitoring.html	**Verificado**
Active Response (official documentation)	https://documentation.wazuh.com/current/user-manual/capabilities/	**Verificado**
MITRE ATT&CK T1485	https://attack.mitre.org/techniques/T1485/	**Verificado**
MITRE ATT&CK T1486	https://attack.mitre.org/techniques/T1486/	**Verificado**
WAZUH-Process-Tree-Viewer	https://github.com/mym0us3r/WAZUH-Process-Tree-Viewer	**Verificado**
Wazuh FIM - Advanced Defender's Handbook - This project	https://github.com/mym0us3r/wazuh-fim-adh	**Este projeto**

Tabela 27 - Referências oficiais e status da verificação

Kislley Rodrigues - Wazuh Ambassador Program
[October 2026 - Wazuh FIM: The Advanced Defender's Handbook]
LinkedIn: https://www.linkedin.com/in/rkbr/
GitHub Repository: https://github.com/mym0us3r/wazuh-fim-adh

Mais informações sobre o Wazuh:

wazuh.

Wazuh - Open Source XDR and SIEM

wazuh.
Ambassadors

Wazuh | Ambassadors Program